



University Admission Management System

Name	ID	Contribution
Sheikh Rubaeid Sanjid	21-45844-3	(33.34%) Query Writing, Scenario Description, Normalization
Abdullah Al Faisal	21-45872-3	(33.33%) UI Interface, Table Creation, Data Insertion
Wasifur Rahaman Chowdhury Alvi	21-45856-3	(33.33%) ER Diagram, Normalization, Table Creation

Course Name: Advance Database Management System

Section: A

Date of Submission: **May 5, 2025**

Introduction:

The University Admission Management System is a project that aims to make the university admission process easier and more efficient. It will help students apply for different programs, submit their personal and academic information, and check the status of their applications. At the same time, it will help university staff manage all the student applications in a smooth and organized way. This system will be built using a database that stores all important data, such as student details, application forms, program offerings, and admission results. The system goal is to reduce the amount of manual work needed, make fewer mistakes while admission, and speed up the admission process. Some Key features of the system are:

- ✓ An online application form for students to fill out and submit.
- ✓ Storing all student and program data in a safe and organized database.
- ✓ Generating reports to check student applications.
- ✓ Allowing students to see the status of their applications.
- ✓ An admin panel for university staff to review and approve applications.

By using this system, both students and staff will benefit from a more efficient and reliable admission process, with easy access to data and better communication throughout the whole process.

User Interface planning:

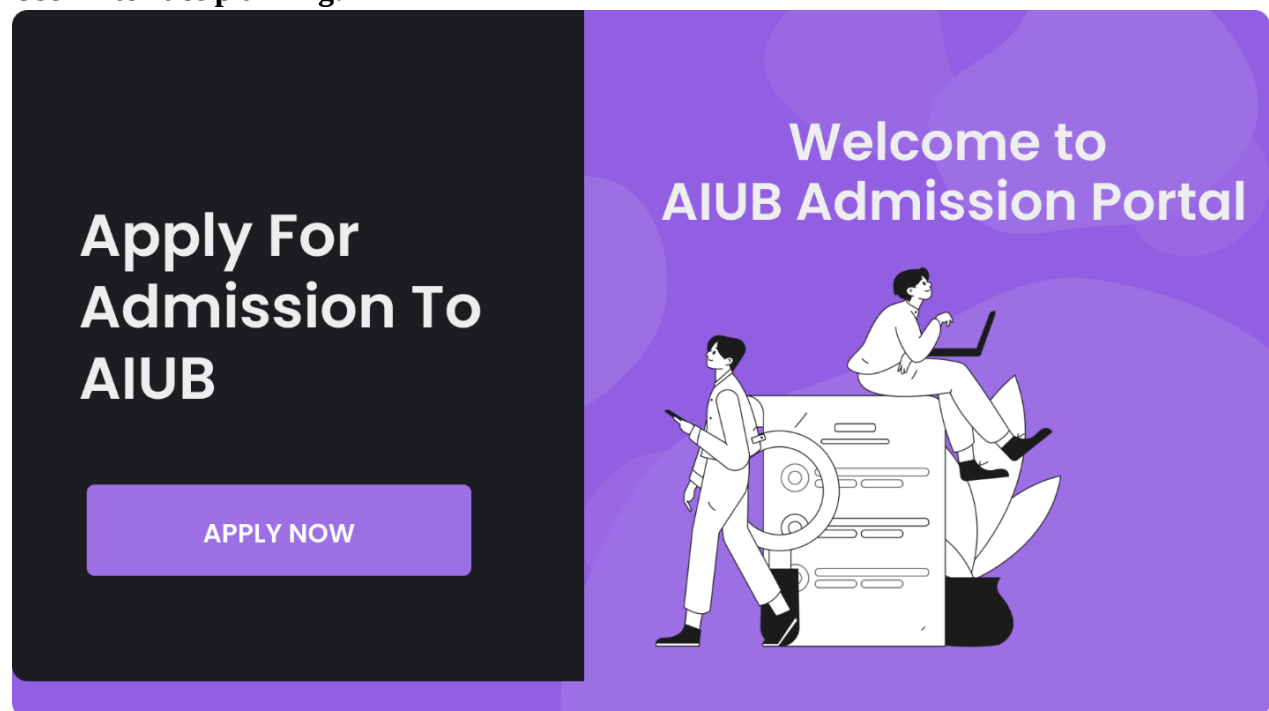


Figure: Landing Page

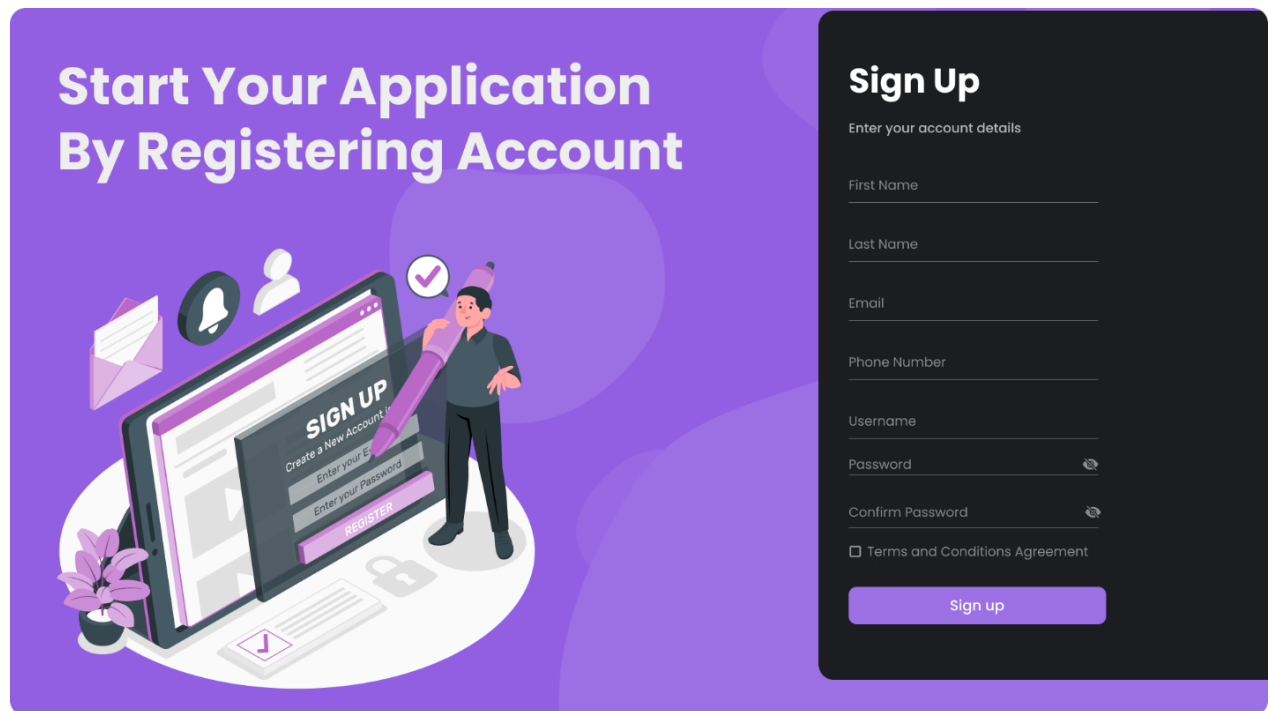


Figure: Signup Page



Figure: Login Page

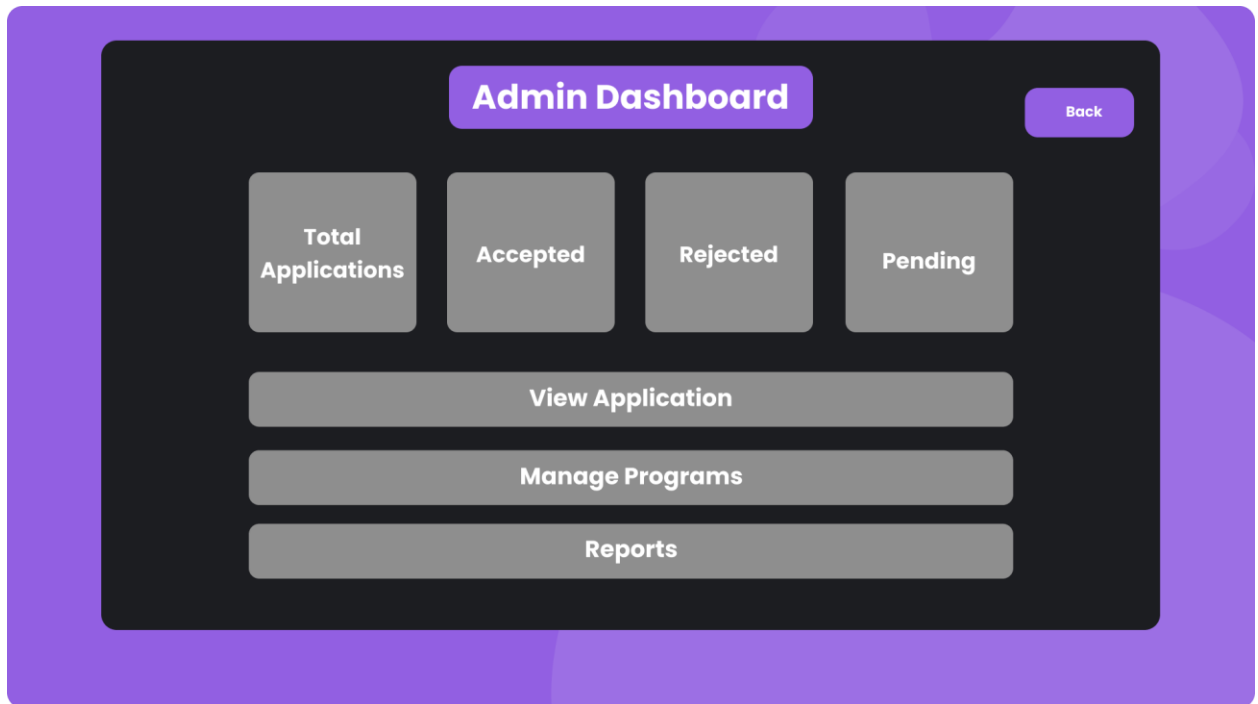


Figure: Admin Panel Page

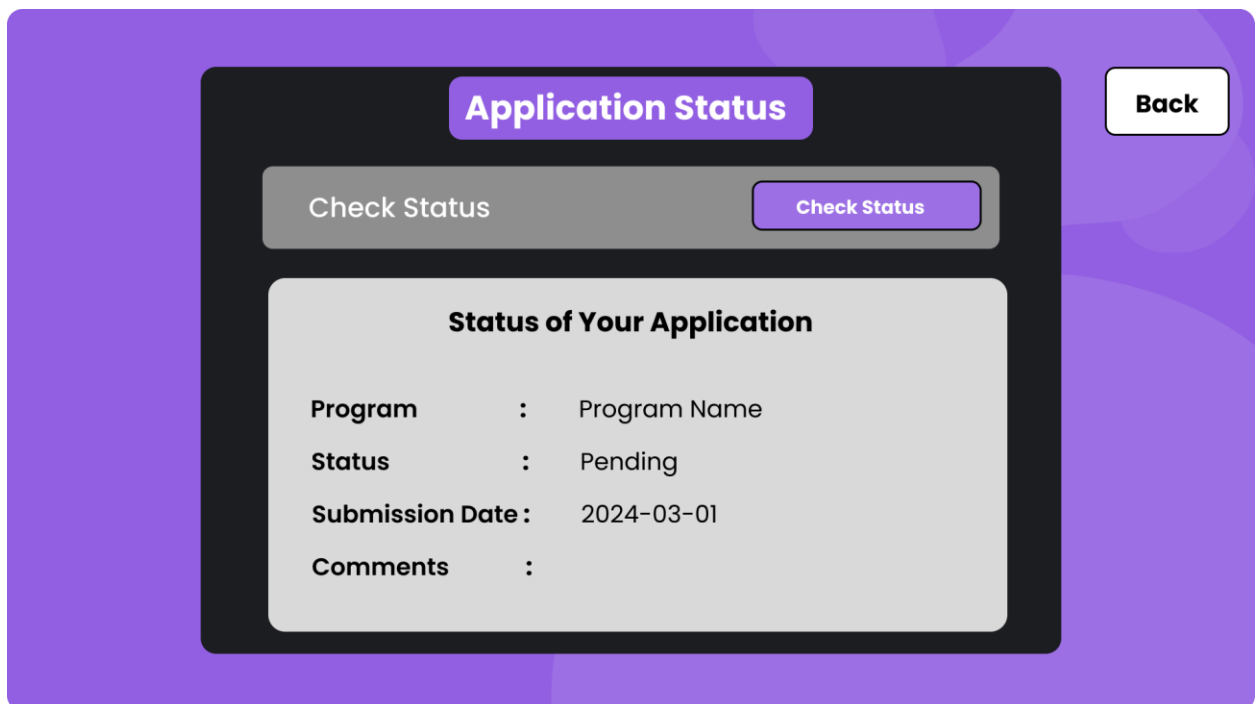


Figure: Application Status Page

Review Application

Back

Student Name	Program	Status	
Name 1	Program 1	Pending	View
Name 2	Program 2	Accepted	View
Name 3	Program 3	Rejected	View
Name 4	Program 4	Accepted	View

Accept

Reject

Figure: Review Application Page

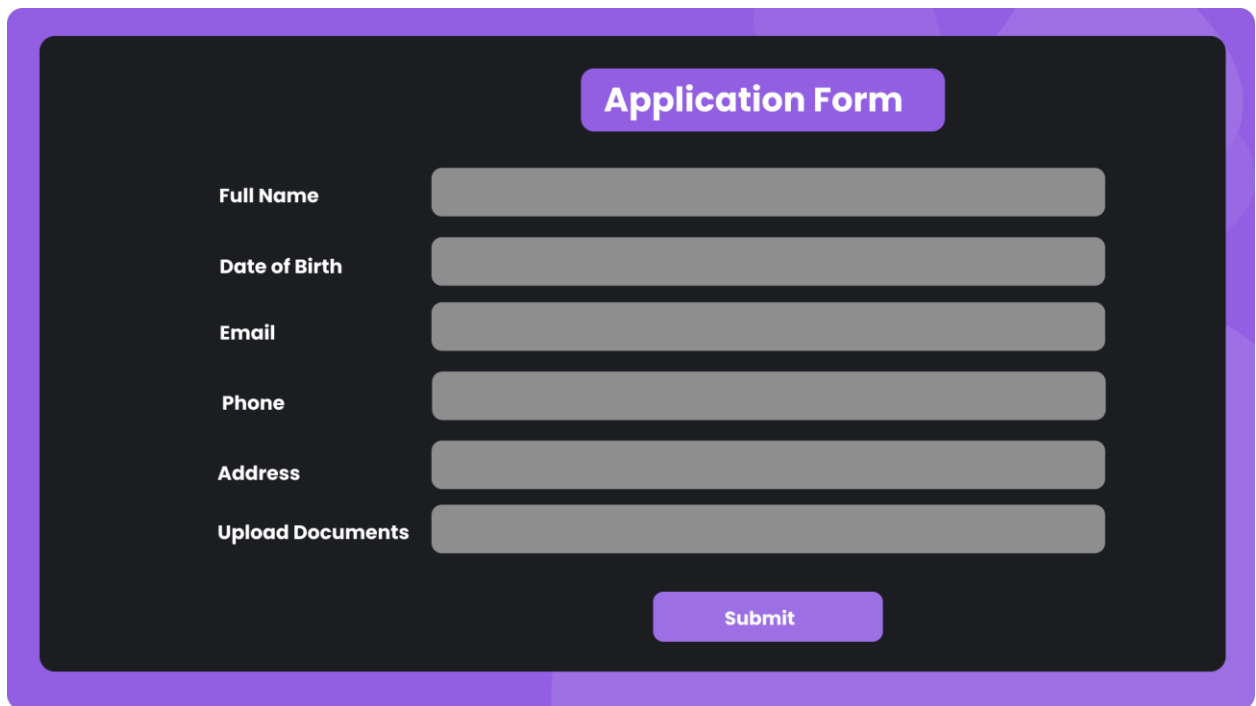
Review Application

Back

Program	Code	Actions
Program 1	Code123	Edit
Program 2	Code456	Edit
Program 3	Code789	Edit
Program 4	Code098	Edit

Add Program

Figure: Review Page



The image shows a web page for an application form. It has a dark blue background with a light blue border. At the top center, there is a light blue button labeled "Application Form". Below this, there are six input fields, each with a label to its left: "Full Name", "Date of Birth", "Email", "Phone", "Address", and "Upload Documents". Each input field is a light gray rectangle. At the bottom center, there is a light blue button labeled "Submit".

Application Form

Full Name

Date of Birth

Email

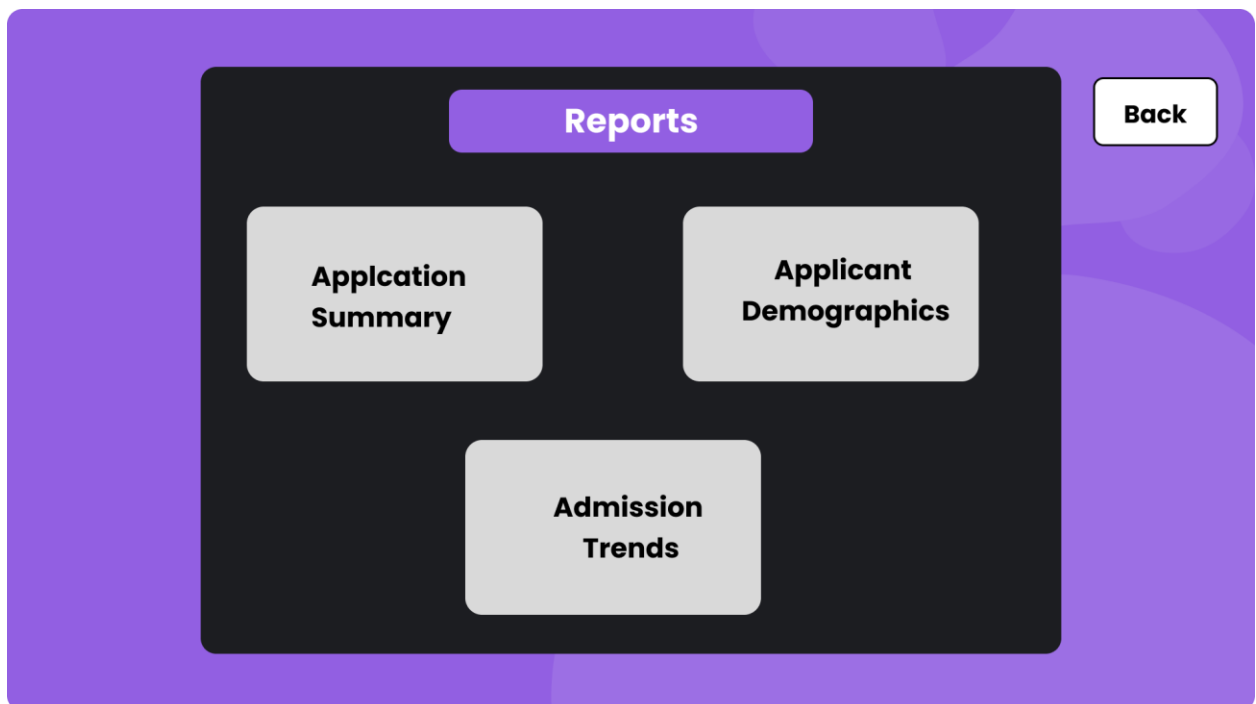
Phone

Address

Upload Documents

Submit

Figure: Application Form Page



The image shows a web page for reports. It has a dark blue background with a light blue border. At the top center, there is a light blue button labeled "Reports". To the right of this button, there is a white button labeled "Back". Below the "Reports" button, there are three light gray boxes arranged in a grid. The top row has two boxes: "Application Summary" on the left and "Applicant Demographics" on the right. The bottom row has one box centered: "Admission Trends".

Reports

Back

Application Summary

Applicant Demographics

Admission Trends

Figure: Reports Page

Scenario Description:

In a university admission management system, a student may apply to multiple academic programs. Each application is submitted by exactly one student. A student is identified by a student ID. The system also stores student name, email, phone number, and address. A student's address is composed of house number, street name, and city.

A program is identified by a unique program code. The system stores the program name, department name, and program level (e.g., Bachelor's, Master's). Each program belongs to exactly one department, and a department must have at least one program.

During the admission process, a student submits an application form. The form records the date of submission, selected program, and any uploaded documents. A student may submit multiple applications to different programs, but only one application per program.

Each application must go through an admission test. The system stores the test ID, test date, total marks, and passing marks. A student can appear in one or more admission tests, and each test result is linked to a specific application.

A student may be accepted or rejected for a program based on test results. The admission result includes application ID, result status (Accepted/Rejected), and comments from the university staff.

Staff members who manage the admission process are identified by a staff ID. The system stores the staff member's name, role (e.g., reviewer, manager), and department. Each application is reviewed and processed by at least one staff member.

The system also stores information on documents submitted by the applicant, such as academic transcripts, ID proof, and certificates. Each document is linked to a specific application.

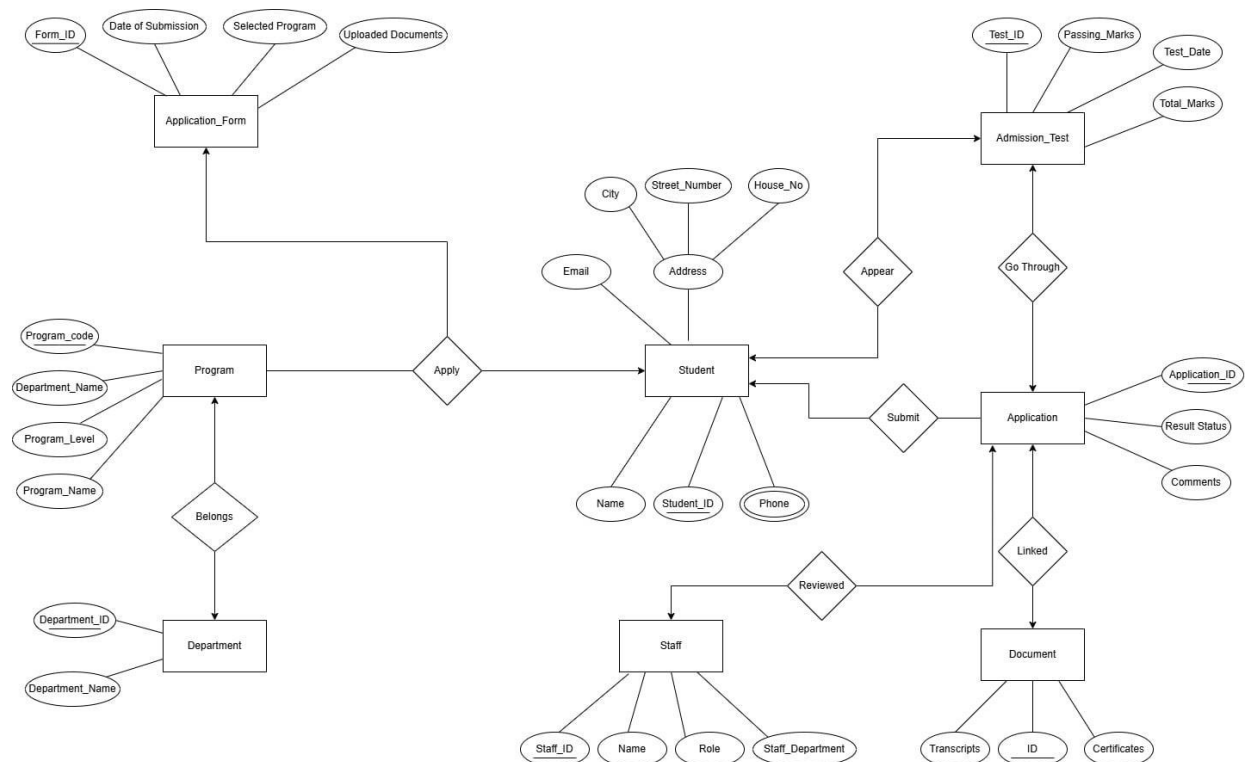
In a university admission management system, a student may apply to multiple academic programs. Each application is submitted by exactly one student. A student is identified by a student ID. The system also stores student name, email, phone number, and address. A student's address is composed of house number, street name, and city.

A program is identified by a unique program code. The system stores the program name, department name, and program level (e.g., Bachelor's, Master's). Each program belongs to exactly one department, and a department must have at least one program.

During the admission process, a student submits an application form. The form records the date of submission, selected program, and any uploaded documents. A student may submit multiple applications to different programs, but only one application per program.

Each application must go through an admission test. The system stores the test ID, test date, total marks, and passing marks. A student can appear in one or more admission tests, and each test result is linked to a specific application.

The system also stores information on documents submitted by the applicant, such as academic transcripts, ID proof, and certificates. Each document is linked to a specific application.



Normalization:

Apply

UNF

Apply (Student_ID, Name, Phone, Email, City, Street_No , House_No, , form_id,
date_of_submission, selected_program, uploaded_documents, program_code,
department_name, program_level, program_name)

1NF

Phone is a multi valued attribute.

1. Student_ID, Name, Phone, Email, City, Street_No , House_No, , form_id,
date_of_submission, selected_program, uploaded_documents, program_code,
department_name, program_level, program_name

2NF

1. Student_ID, Name, Phone, Email, City, Street_No , House_No,
2. form_id, date_of_submission, selected_program, uploaded_documents,
3. program_code, department_name, program_level, program_name
4. SFP_Id, Student_ID, form_id, program_code

3NF

1. Student_ID, Name, Phone, Email, City, Street_No , House_No,
2. form_id, date_of_submission, selected_program, uploaded_documents,
3. program_code, program_level, DepPro_Id (FK)
4. SFP_Id, Student_ID, form_id, program_code
5. DepPro_Id, department_name, program_name

Table Creation

1. Student_ID, Name, Phone, Email, City, Street_No , House_No,
2. form_id, date_of_submission, selected_program, uploaded_documents,
3. program_code, program_level, DepPro_Id (FK)
4. SFP_Id, Student_ID, form_id, program_code
5. DepPro_Id, department_name, program_name

Belongs

UNF

Belongs (program_code, department_name, program_level, program_name, department_id,
department_name)

1NF

There is no multi valued attribute. Relation already in 1NF.

program_code, department_name, program_level, program_name, department_id,
department_name

2NF

1. program_code, department_name, program_level, program_name
2. department_id, department_name
3. DP_Id, program_code, department_id

3NF

1. program_code, program_level, DepPro_Id (FK)
2. department_id, department_name
3. DP_Id, program_code, department_id
4. DepPro_Id, department_name, program_name

Table Creation

1. program_code, program_level, DepPro_Id (FK)
2. department_id, department_name
3. DP_Id, program_code, department_id
4. DepPro_Id, department_name, program_name

Appear

UNF

Appear (student_id, name, email, phone, city, street_number, house_no, test_id,
passing_marks, test_date, total_marks)

1NF

Phone is a multi valued attribute.

1. student_id, name, email, phone, city, street_number, house_no, test_id, passing_marks,
test_date, total_marks

2NF

1. student_id, name, email, phone, city, street_number, house_no
2. test_id, passing_marks, test_date, total_marks
3. ST_Id, student_id, test_id

3NF

1. student_id, name, email, phone, city, street_number, house_no

2. test_id, test_date, PT_Id (FK)
3. ST_Id, student_id, test_id
4. PT_Id, passing_marks, total_marks

Table Creation

1. student_id, name, email, phone, city, street_number, house_no
2. test_id, test_date, PT_Id (FK)
3. ST_Id, student_id, test_id
4. PT_Id, passing_marks, total_marks

Go Through

UNF

Go Through (application_id, result_status, comments, test_id, passing_marks, test_date, total_marks)

1NF

There is no multi valued attribute. Relation already in 1NF.

1. application_id, result_status, comments, test_id, passing_marks, test_date, total_marks

2NF

1. application_id, result_status, comments,
2. test_id, passing_marks, test_date, total_marks
3. AT_Id, application_id, test_id,

3NF

1. application_id, result_status, comments,
2. test_id, test_date, total_marks
3. AT_Id, application_id, test_id,
4. PT_Id, passing_marks, total_marks

Table Creation

1. application_id, result_status, comments,
2. test_id, test_date, total_marks
3. AT_Id, application_id, test_id,
4. PT_Id, passing_marks, total_marks

Submit

UNF

Submit (application_id, result_status, comments, student_id, name, email, phone, address, city, street_number, house_no)

1NF

Phone is a multi valued attribute.

1. application_id, result_status, comments, student_id, name, email, phone, address, city, street_number, house_no

2NF

1. application_id, result_status, comments
2. student_id, name, email, phone, address, city, street_number, house_no
3. AS_Id, application_id, student_id,

3NF

There is no transitive dependency. Relation already in 3NF

1. application_id, result_status, comments
2. student_id, name, email, phone, address, city, street_number, house_no
3. AS_Id, application_id, student_id,

Table Creation

1. application_id, result_status, comments
2. student_id, name, email, phone, address, city, street_number, house_no
3. AS_Id, application_id, student_id,

Linked

UNF

Linked (application_id, date_of_submission, selected_program, uploaded_documents, document_id, transcripts, certificates)

1NF

There is no multi valued attribute. Relation already in 1NF.

1. application_id, date_of_submission, selected_program, uploaded_documents, document_id, transcripts, certificates

2NF

1. application_id, date_of_submission, selected_program, uploaded_documents,
2. document_id, transcripts, certificates
3. AD_Id, application_id, document_id,

3NF

There is no transitive dependency. Relation already in 3NF

1. application_id, date_of_submission, selected_program, uploaded_documents,
2. document_id, transcripts, certificates
3. AD_Id, application_id, document_id,

Table Creation

1. application_id, date_of_submission, selected_program, uploaded_documents,
2. document_id, transcripts, certificates
3. AD_Id, application_id, document_id,

Reviewed

UNF

Reviewed (application_id, result_status, comments, staff_id, name, role, staff_department)

1NF

There is no multi valued attribute. Relation already in 1NF.

1. application_id, result_status, comments, staff_id, name, role, staff_department

2NF

1. application_id, result_status, comments
2. staff_id, name, role, staff_department
3. AS_Id, application_id, staff_id

3NF

There is no transitive dependency. Relation already in 3NF

1. application_id, result_status, comments
2. staff_id, name, role, staff_department
3. AS_Id, application_id, staff_id

Table Creation

1. application_id, result_status, comments
2. staff_id, name, role, staff_department
3. AS_Id, application_id, staff_id

Temporary Tables

1. Student_ID, Name, Phone, Email, City, Street_No , House_No,
2. form_id, date_of_submission, selected_program, uploaded_documents,
3. program_code, program_level, DepPro_Id (FK)

4. SFP_Id, Student_ID, form_id, program_code
5. DepPro_Id, department_name, program_name
6. program_code, program_level, DepPro_Id (FK)
7. department_id, department_name
8. DP_Id, program_code, department_id
9. DepPro_Id, department_name, program_name
10. student_id, name, email, phone, city, street_no, house_no
11. test_id, test_date, PT_Id (FK)
12. ST_Id, student_id, test_id
13. PT_Id, passing_marks, total_marks
14. application_id, result_status, comments,
15. test_id, test_date, total_marks
16. AT_Id, application_id, test_id,
17. PT_Id, passing_marks, total_marks
18. application_id, result_status, comments
19. student_id, name, email, phone, city, street_number, house_no
20. AS_Id, application_id, student_id,
21. application_id, date_of_submission, selected_program, uploaded_documents,
22. document_id, transcripts, certificates
23. AD_Id, application_id, document_id,
24. application_id, result_status, comments
25. staff_id, name, role, staff_department
26. AS_Id, application_id, staff_id

Final Tables

1. Student_ID, Name, Phone, Email, City, Street_No , House_No,
2. form_id, date_of_submission, selected_program, uploaded_documents,
3. program_code, program_level, DepPro_Id (FK)
4. SFP_Id, Student_ID, form_id, program_code
5. DepPro_Id, department_name, program_name
6. department_id, department_name
7. DP_Id, program_code, department_id
8. test_id, test_date, PT_Id (FK)

9. ST_Id, student_id, test_id
10. PT_Id, passing_marks, total_marks
11. application_id, result_status, comments,
12. test_id, test_date, total_marks
13. AT_Id, application_id, test_id,
14. PT_Id, passing_marks, total_marks
15. application_id, result_status, comments
16. AS_Id, application_id, student_id,
17. application_id, date_of_submission, selected_program, uploaded_documents,
18. document_id, transcripts, certificates
19. AD_Id, application_id, document_id,
20. staff_id, name, role, staff_department
21. AS_Id, application_id, staff_id

Table Creation:

Table Name: Test Table

```
CREATE TABLE Test (  
    test_id NUMBER PRIMARY KEY,  
    test_date DATE,  
    PT_Id NUMBER,  
    FOREIGN KEY (PT_Id) REFERENCES PassingTest(PT_Id)  
);
```

The screenshot shows the Oracle Database Express Edition SQL Commands window. The SQL script executed is as follows:

```
CREATE TABLE Test (  
    test_id NUMBER PRIMARY KEY,  
    test_date DATE,  
    PT_Id NUMBER,  
    FOREIGN KEY (PT_Id) REFERENCES PassingTest(PT_Id)  
);  
  
INSERT INTO Test (test_id, test_date, PT_Id) VALUES (1, TO_DATE('2025-04-01', 'YYYY-MM-DD'), 1)  
INSERT INTO Test (test_id, test_date, PT_Id) VALUES (2, TO_DATE('2025-04-02', 'YYYY-MM-DD'), 2)  
INSERT INTO Test (test_id, test_date, PT_Id) VALUES (3, TO_DATE('2025-04-03', 'YYYY-MM-DD'), 3)  
INSERT INTO Test (test_id, test_date, PT_Id) VALUES (4, TO_DATE('2025-04-04', 'YYYY-MM-DD'), 4)  
INSERT INTO Test (test_id, test_date, PT_Id) VALUES (5, TO_DATE('2025-04-05', 'YYYY-MM-DD'), 5)  
select * from Test
```

The results of the query are displayed in a table with 5 rows:

TEST_ID	TEST_DATE	PT_ID
1	01-APR-25	1
2	02-APR-25	2
3	03-APR-25	3
4	04-APR-25	4
5	05-APR-25	5

5 rows returned in 0.01 seconds

Table Name: Passing Test CREATE

```
TABLE PassingTest (  
    PT_Id NUMBER PRIMARY KEY,  
    passing_marks NUMBER,  
    total_marks NUMBER  
);
```

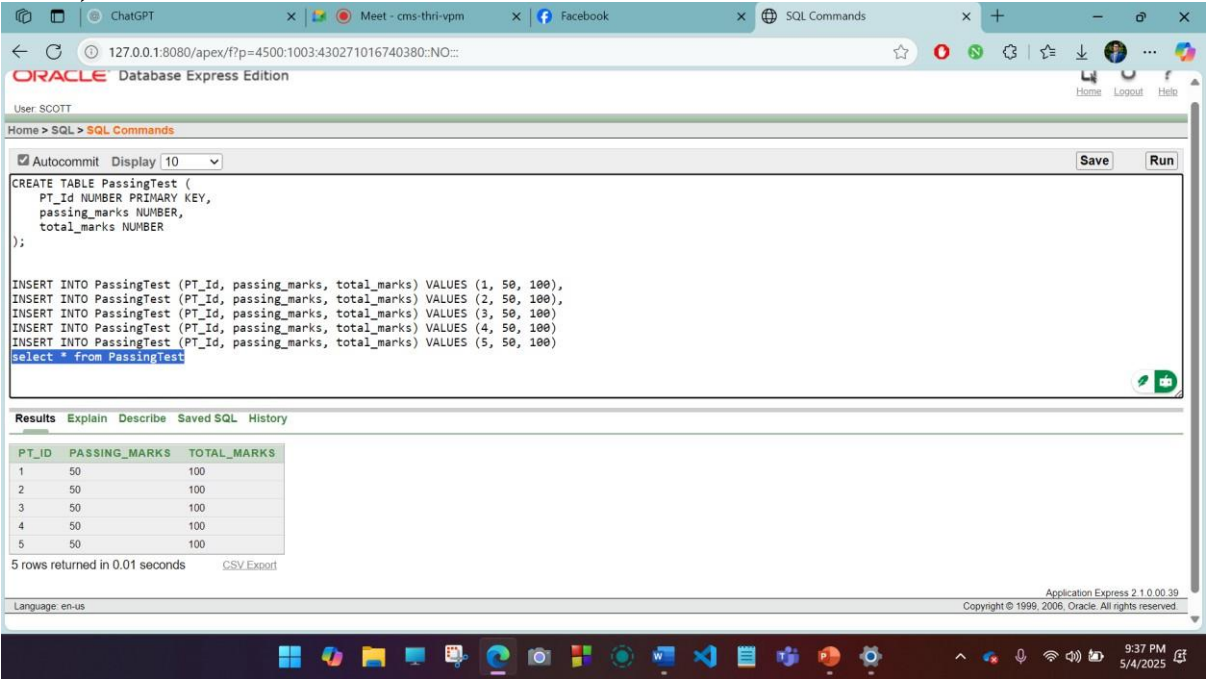
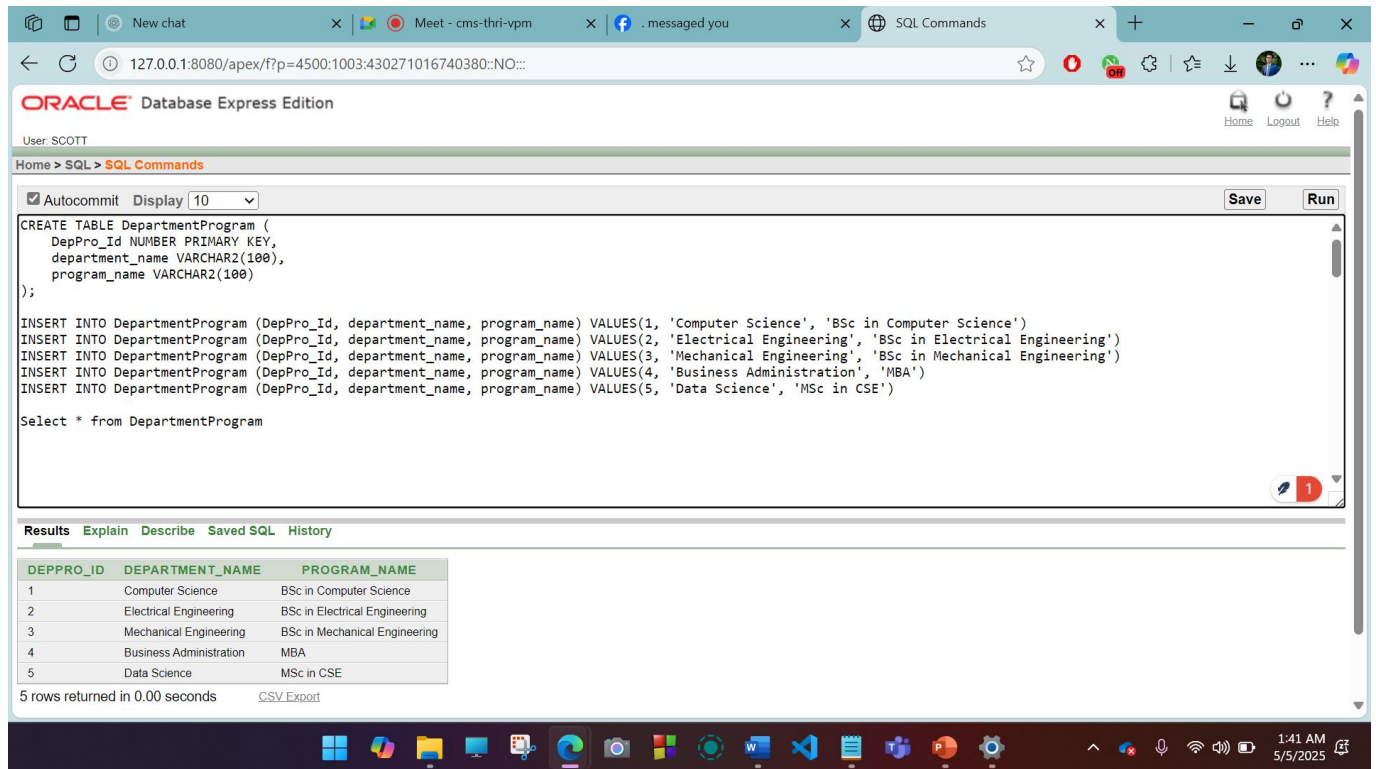


Table Name: DepartmentProgram

```
CREATE TABLE DepartmentProgram (
  DepPro_Id NUMBER PRIMARY KEY,
  department_name VARCHAR2(100),
  program_name VARCHAR2(100)
);
```



The screenshot shows the Oracle Database Express Edition interface. The SQL Commands window contains the following code:

```
CREATE TABLE DepartmentProgram (
  DepPro_Id NUMBER PRIMARY KEY,
  department_name VARCHAR2(100),
  program_name VARCHAR2(100)
);

INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(1, 'Computer Science', 'BSc in Computer Science')
INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(2, 'Electrical Engineering', 'BSc in Electrical Engineering')
INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(3, 'Mechanical Engineering', 'BSc in Mechanical Engineering')
INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(4, 'Business Administration', 'MBA')
INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(5, 'Data Science', 'MSc in CSE')

Select * from DepartmentProgram
```

The Results window shows the following data:

DEPPRO_ID	DEPARTMENT_NAME	PROGRAM_NAME
1	Computer Science	BSc in Computer Science
2	Electrical Engineering	BSc in Electrical Engineering
3	Mechanical Engineering	BSc in Mechanical Engineering
4	Business Administration	MBA
5	Data Science	MSc in CSE

5 rows returned in 0.00 seconds

Table Name: StudentTest

```
CREATE TABLE StudentTest (
  ST_Id NUMBER PRIMARY KEY,
  student_id NUMBER,
  test_id NUMBER,
  FOREIGN KEY (student_id) REFERENCES Student(Student_ID),
  FOREIGN KEY (test_id) REFERENCES Test(test_id)
);
```

Oracle Database Express Edition - User: SCOTT

SQL Commands

Autocommit: ☒ Display: 10

Save Run

```

CREATE TABLE StudentTest (
  ST_Id NUMBER PRIMARY KEY,
  student_id NUMBER,
  test_id NUMBER,
  FOREIGN KEY (student_id) REFERENCES Student(Student_ID),
  FOREIGN KEY (test_id) REFERENCES Test(test_id)
);

INSERT INTO StudentTest (ST_Id, student_id, test_id) VALUES(1, 1, 1)
INSERT INTO StudentTest (ST_Id, student_id, test_id) VALUES(2, 2, 2)
INSERT INTO StudentTest (ST_Id, student_id, test_id) VALUES(3, 3, 3)
INSERT INTO StudentTest (ST_Id, student_id, test_id) VALUES(4, 4, 4)
INSERT INTO StudentTest (ST_Id, student_id, test_id) VALUES(5, 5, 5)
select * from StudentTest
  
```

Results Explain Describe Saved SQL History

ST_ID	STUDENT_ID	TEST_ID
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5

5 rows returned in 0.00 seconds [CSV Export](#)

Language: en-us Application Express 2.1.0.00.39 Copyright © 1999, 2006, Oracle. All rights reserved.

Table Name: Application

```

CREATE TABLE Application (
  application_id NUMBER PRIMARY KEY,
  result_status VARCHAR2(50),
  comments VARCHAR2(200)
);
  
```

Oracle Database Express Edition - User: SCOTT

SQL Commands

Autocommit: ☒ Display: 10

Save Run

```

CREATE TABLE Application (
  application_id NUMBER PRIMARY KEY,
  result_status VARCHAR2(50),
  comments VARCHAR2(200)
);

INSERT INTO Application (application_id, result_status, comments) VALUES(1, 'Accepted', 'Congratulations on being accepted!')
INSERT INTO Application (application_id, result_status, comments) VALUES(2, 'Rejected', 'Application did not meet criteria.')
INSERT INTO Application (application_id, result_status, comments) VALUES(3, 'Accepted', 'Congratulations on being accepted!')
INSERT INTO Application (application_id, result_status, comments) VALUES(4, 'Under Review', 'Waiting for final decision.')
INSERT INTO Application (application_id, result_status, comments) VALUES(5, 'Accepted', 'Congratulations on being accepted!')
select * from Application
  
```

Results Explain Describe Saved SQL History

APPLICATION_ID	RESULT_STATUS	COMMENTS
1	Accepted	Congratulations on being accepted!
2	Rejected	Application did not meet criteria.
3	Accepted	Congratulations on being accepted!
4	Under Review	Waiting for final decision.
5	Accepted	Congratulations on being accepted!

5 rows returned in 0.00 seconds [CSV Export](#)

Language: en-us Application Express 2.1.0.00.39 Copyright © 1999, 2006, Oracle. All rights reserved.

Table Name: Staff

```
CREATE TABLE Staff (  
    staff_id NUMBER PRIMARY KEY,  
    name VARCHAR2(100),  
    role VARCHAR2(50),  
    staff_department VARCHAR2(100)  
);
```

The screenshot shows the Oracle Database Express Edition web interface. The browser address bar displays the URL: 127.0.0.1:8080/apex/?p=4500:1003:430271016740380:NO... The page title is "ORACLE Database Express Edition". The user is logged in as "User: SCOTT". The "SQL Commands" tab is active, showing the following SQL code:

```
CREATE TABLE Staff (  
    staff_id NUMBER PRIMARY KEY,  
    name VARCHAR2(100),  
    role VARCHAR2(50),  
    staff_department VARCHAR2(100)  
);  
  
INSERT INTO Staff (staff_id, name, role, staff_department) VALUES(1, 'Alice Johnson', 'Administrator', 'Admissions')  
INSERT INTO Staff (staff_id, name, role, staff_department) VALUES(2, 'Bob Williams', 'Professor', 'Computer Science')  
INSERT INTO Staff (staff_id, name, role, staff_department) VALUES(3, 'Charlie Brown', 'Lecturer', 'Electrical Engineering')  
INSERT INTO Staff (staff_id, name, role, staff_department) VALUES(4, 'Diana Smith', 'Professor', 'Mechanical Engineering')  
INSERT INTO Staff (staff_id, name, role, staff_department) VALUES(5, 'Edward Clark', 'Lecturer', 'Data Science')  
select * from Staff
```

The "Results" tab is active, displaying the following table:

STAFF_ID	NAME	ROLE	STAFF_DEPARTMENT
1	Alice Johnson	Administrator	Admissions
2	Bob Williams	Professor	Computer Science
3	Charlie Brown	Lecturer	Electrical Engineering
4	Diana Smith	Professor	Mechanical Engineering
5	Edward Clark	Lecturer	Data Science

Below the table, it says "5 rows returned in 0.03 seconds" and provides a "CSV Export" link. The footer of the interface shows "Application Express 2.1.0.00.39" and "Copyright © 1999, 2006, Oracle. All rights reserved."

Table Name: ApplicationDocument

```
CREATE TABLE ApplicationDocument (  
    AD_Id NUMBER PRIMARY KEY,  
    application_id NUMBER,  
    document_id NUMBER,  
    FOREIGN KEY (application_id) REFERENCES Application(application_id),  
    FOREIGN KEY (document_id) REFERENCES Document(document_id)  
);
```

Oracle Database Express Edition interface showing SQL Commands and Results.

User: SCOTT

Home > SQL > SQL Commands

Autocommit: ☒ Display: 10

Save Run

```

CREATE TABLE ApplicationDocument (
  AD_Id NUMBER PRIMARY KEY,
  application_id NUMBER,
  document_id NUMBER,
  FOREIGN KEY (application_id) REFERENCES Application(application_id),
  FOREIGN KEY (document_id) REFERENCES Document(document_id)
);

INSERT INTO ApplicationDocument (AD_Id, application_id, document_id) VALUES(1, 1, 1)
INSERT INTO ApplicationDocument (AD_Id, application_id, document_id) VALUES(2, 2, 2)
INSERT INTO ApplicationDocument (AD_Id, application_id, document_id) VALUES(3, 3, 3)
INSERT INTO ApplicationDocument (AD_Id, application_id, document_id) VALUES(4, 4, 4)
INSERT INTO ApplicationDocument (AD_Id, application_id, document_id) VALUES(5, 5, 5)
select * from ApplicationDocument
  
```

Results Explain Describe Saved SQL History

AD_ID	APPLICATION_ID	DOCUMENT_ID
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5

5 rows returned in 0.01 seconds [CSV Export](#)

Language: en-us

Application Express 2.1.0.00.39
Copyright © 1999, 2006, Oracle. All rights reserved.

Table Name: Document

```

CREATE TABLE Document (
  document_id NUMBER PRIMARY KEY,
  transcripts VARCHAR2(200),
  certificates VARCHAR2(200)
);
  
```

Oracle Database Express Edition interface showing SQL Commands and Results.

User: SCOTT

Home > SQL > SQL Commands

Autocommit: ☒ Display: 10

Save Run

```

CREATE TABLE Document (
  document_id NUMBER PRIMARY KEY,
  transcripts VARCHAR2(200),
  certificates VARCHAR2(200)
);

INSERT INTO Document (document_id, transcripts, certificates) VALUES(1, 'Transcript 1', 'Certificate 1')
INSERT INTO Document (document_id, transcripts, certificates) VALUES(2, 'Transcript 2', 'Certificate 2')
INSERT INTO Document (document_id, transcripts, certificates) VALUES(3, 'Transcript 3', 'Certificate 3')
INSERT INTO Document (document_id, transcripts, certificates) VALUES(4, 'Transcript 4', 'Certificate 4')
INSERT INTO Document (document_id, transcripts, certificates) VALUES(5, 'Transcript 5', 'Certificate 5')
select * from Document
  
```

Results Explain Describe Saved SQL History

DOCUMENT_ID	TRANSCRIPTS	CERTIFICATES
1	Transcript 1	Certificate 1
2	Transcript 2	Certificate 2
3	Transcript 3	Certificate 3
4	Transcript 4	Certificate 4
5	Transcript 5	Certificate 5

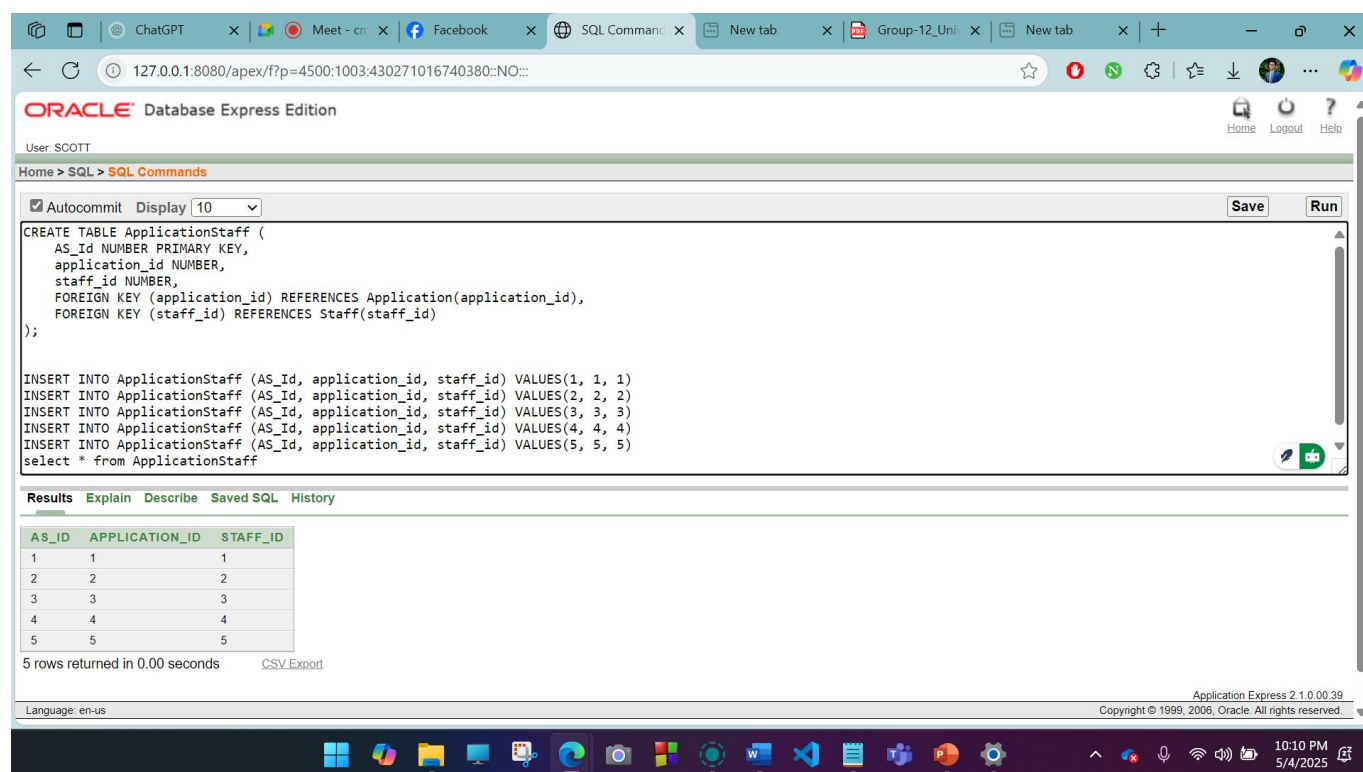
5 rows returned in 0.00 seconds [CSV Export](#)

Language: en-us

Application Express 2.1.0.00.39
Copyright © 1999, 2006, Oracle. All rights reserved.

Table Name: ApplicationStaff

```
CREATE TABLE ApplicationStaff (  
    AS_Id NUMBER PRIMARY KEY,  
    application_id NUMBER,  
    staff_id NUMBER,  
    FOREIGN KEY (application_id) REFERENCES Application(application_id),  
    FOREIGN KEY (staff_id) REFERENCES Staff(staff_id)  
);
```



The screenshot shows the Oracle Database Express Edition SQL Command window. The user is SCOTT. The SQL Command window contains the following code:

```
CREATE TABLE ApplicationStaff (  
    AS_Id NUMBER PRIMARY KEY,  
    application_id NUMBER,  
    staff_id NUMBER,  
    FOREIGN KEY (application_id) REFERENCES Application(application_id),  
    FOREIGN KEY (staff_id) REFERENCES Staff(staff_id)  
);  
  
INSERT INTO ApplicationStaff (AS_Id, application_id, staff_id) VALUES(1, 1, 1)  
INSERT INTO ApplicationStaff (AS_Id, application_id, staff_id) VALUES(2, 2, 2)  
INSERT INTO ApplicationStaff (AS_Id, application_id, staff_id) VALUES(3, 3, 3)  
INSERT INTO ApplicationStaff (AS_Id, application_id, staff_id) VALUES(4, 4, 4)  
INSERT INTO ApplicationStaff (AS_Id, application_id, staff_id) VALUES(5, 5, 5)  
select * from ApplicationStaff
```

The Results tab shows the following data:

AS_ID	APPLICATION_ID	STAFF_ID
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5

5 rows returned in 0.00 seconds

Table Name: ApplicationTest

```
CREATE TABLE ApplicationTest (  
    AT_Id NUMBER PRIMARY KEY,  
    application_id NUMBER,  
    test_id NUMBER,  
    FOREIGN KEY (application_id) REFERENCES Application(application_id),  
    FOREIGN KEY (test_id) REFERENCES Test(test_id)  
);
```


Oracle Database Express Edition interface showing SQL commands and results.

User: SCOTT

Home > SQL > SQL Commands

Autocommit: ☒ Display: 10

Save Run

```

CREATE TABLE ApplicationTest (
  AT_Id NUMBER PRIMARY KEY,
  application_id NUMBER,
  test_id NUMBER,
  FOREIGN KEY (application_id) REFERENCES Application(application_id),
  FOREIGN KEY (test_id) REFERENCES Test(test_id)
);

INSERT INTO ApplicationTest (AT_Id, application_id, test_id) VALUES(1, 1, 1)
INSERT INTO ApplicationTest (AT_Id, application_id, test_id) VALUES(2, 2, 2)
INSERT INTO ApplicationTest (AT_Id, application_id, test_id) VALUES(3, 3, 3)
INSERT INTO ApplicationTest (AT_Id, application_id, test_id) VALUES(4, 4, 4)
INSERT INTO ApplicationTest (AT_Id, application_id, test_id) VALUES(5, 5, 5)
select * from ApplicationTest
  
```

Results Explain Describe Saved SQL History

AT_ID	APPLICATION_ID	TEST_ID
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5

5 rows returned in 0.02 seconds [CSV Export](#)

Language: en-us

Application Express 2.1.0.00.39
Copyright © 1999, 2006, Oracle. All rights reserved.

Table Name: StaffAssignment Table

CREATE TABLE StaffAssignment (

StaffAssignment_Id NUMBER PRIMARY KEY,

staff_id NUMBER,

department_id NUMBER,

FOREIGN KEY (staff_id) REFERENCES Staff(staff_id),

FOREIGN KEY (department_id) REFERENCES Department(department_id)

);

Oracle Database Express Edition interface showing SQL commands and results.

User: SCOTT

Home > SQL > SQL Commands

Autocommit: ☒ Display: 10

Save Run

```

CREATE TABLE StaffAssignment (
  StaffAssignment_Id NUMBER PRIMARY KEY,
  staff_id NUMBER,
  department_id NUMBER,
  FOREIGN KEY (staff_id) REFERENCES Staff(staff_id),
  FOREIGN KEY (department_id) REFERENCES Department(department_id)
);

INSERT INTO StaffAssignment (StaffAssignment_Id, staff_id, department_id) VALUES
(5, 5, 5);

select * from StaffAssignment
  
```

Results Explain Describe Saved SQL History

STAFFASSIGNMENT_ID	STAFF_ID	DEPARTMENT_ID
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5

5 rows returned in 0.00 seconds [CSV Export](#)

Language: en-us

Application Express 2.1.0.00.39
Copyright © 1999, 2006, Oracle. All rights reserved.

Table Name: Form

```
CREATE TABLE Form (  
    form_id NUMBER PRIMARY KEY,  
    date_of_submission DATE,  
    selected_program VARCHAR2(100),  
    uploaded_documents VARCHAR2(200)  
);
```

The screenshot shows the Oracle Database Express Edition interface. The SQL Commands window contains the following code:

```
CREATE TABLE Form (  
    form_id NUMBER PRIMARY KEY,  
    date_of_submission DATE,  
    selected_program VARCHAR2(100),  
    uploaded_documents VARCHAR2(200)  
);  
INSERT INTO Form (form_id, date_of_submission, selected_program, uploaded_documents) VALUES  
(5, TO_DATE('2025-04-05', 'YYYY-MM-DD'), 'Data Science', 'Transcript, Research Paper');  
Select * from Form
```

The Results window shows the following data:

FORM_ID	DATE_OF_SUBMISSION	SELECTED_PROGRAM	UPLOADED_DOCUMENTS
1	01-APR-25	Computer Science	Transcript, CV
2	02-APR-25	Electrical Engineering	Transcript, Recommendation Letter
3	03-APR-25	Mechanical Engineering	Transcript, Portfolio
4	04-APR-25	Business Administration	Transcript, Personal Statement
5	05-APR-25	Data Science	Transcript, Research Paper

5 rows returned in 0.00 seconds

Table Name: Student

```
CREATE TABLE Student (  
    Student_ID NUMBER PRIMARY KEY,  
    Name VARCHAR2(100),  
    Phone VARCHAR2(15),  
    Email VARCHAR2(100),  
    City VARCHAR2(50),  
    Street_No VARCHAR2(20),  
    House_No VARCHAR2(20)  
);
```

Oracle Database Express Edition

User: SCOTT

Home > SQL > SQL Commands

Autocommit Display: 10

Save Run

```

CREATE TABLE Student (
  Student_ID NUMBER PRIMARY KEY,
  Name VARCHAR2(100),
  Phone VARCHAR2(15),
  Email VARCHAR2(100),
  City VARCHAR2(50),
  Street_No VARCHAR2(20),
  House_No VARCHAR2(20)
);
INSERT INTO Student (Student_ID, Name, Phone, Email, City, Street_No, House_No) VALUES
(5, 'Mehdi Hasan', '3456345634', 'mehdi.hasan@example.com', 'Barisal', 'Street 5', 'House 105')
Select * from Student

```

Results Explain Describe Saved SQL History

STUDENT_ID	NAME	PHONE	EMAIL	CITY	STREET_NO	HOUSE_NO
1	John Doe	1234567890	john.doe@example.com	Dhaka	Street 1	House 101
2	Jane Smith	0987654321	jane.smith@example.com	Chittagong	Street 2	House 102
3	Ali Khan	1234123412	ali.khan@example.com	Sylhet	Street 3	House 103
4	Sara Ali	2345234523	sara.ali@example.com	Rajshahi	Street 4	House 104
5	Mehdi Hasan	3456345634	mehdi.hasan@example.com	Barisal	Street 5	House 105

5 rows returned in 0.00 seconds CSV Export

Application Express 2.1.0.00.39

Copyright © 1999, 2006, Oracle. All rights reserved.

Language: en-us

28°C Haze 11:39 PM 5/4/2025

Table Name: Department Table

```

CREATE TABLE Department (
  department_id NUMBER PRIMARY KEY,
  department_name VARCHAR2(100)
);

```

Oracle Database Express Edition

User: SCOTT

Home > SQL > SQL Commands

Autocommit Display: 10

Save Run

```

CREATE TABLE Department (
  department_id NUMBER PRIMARY KEY,
  department_name VARCHAR2(100)
);
INSERT INTO Department (department_id, department_name) VALUES
(5, 'Data Science');
Select * from Department

```

Results Explain Describe Saved SQL History

DEPARTMENT_ID	DEPARTMENT_NAME
1	Computer Science
2	Electrical Engineering
3	Mechanical Engineering
4	Business Administration
5	Data Science

5 rows returned in 0.00 seconds CSV Export

Application Express 2.1.0.00.39

Copyright © 1999, 2006, Oracle. All rights reserved.

Language: en-us

28°C Haze 11:48 PM 5/4/2025

Table Name: DePro_Assignment

```

CREATE TABLE DepPro_Assignment (
  DP_Id NUMBER PRIMARY KEY,
  program_code VARCHAR2(50),
  department_id NUMBER,
  FOREIGN KEY (program_code) REFERENCES Program(program_code),
  FOREIGN KEY (department_id) REFERENCES Department(department_id)
);

```


Oracle Database Express Edition interface showing SQL Commands and Results.

User: SCOTT

SQL Commands:

```

CREATE TABLE DepPro_Assignment (
  DP_Id NUMBER PRIMARY KEY,
  program_code VARCHAR2(50),
  department_id NUMBER,
  FOREIGN KEY (program_code) REFERENCES Program(program_code),
  FOREIGN KEY (department_id) REFERENCES Department(department_id)
);

INSERT INTO DepPro_Assignment (DP_Id, program_code, department_id) VALUES(1, 'CS101', 1)
INSERT INTO DepPro_Assignment (DP_Id, program_code, department_id) VALUES(2, 'EE102', 2)
INSERT INTO DepPro_Assignment (DP_Id, program_code, department_id) VALUES(3, 'ME103', 3)
INSERT INTO DepPro_Assignment (DP_Id, program_code, department_id) VALUES(4, 'BA104', 4)
INSERT INTO DepPro_Assignment (DP_Id, program_code, department_id) VALUES(5, 'DS105', 5)

select * from DepPro_Assignment
  
```

Results:

DP_ID	PROGRAM_CODE	DEPARTMENT_ID
1	CS101	1
2	EE102	2
3	ME103	3
4	BA104	4
5	DS105	5

5 rows returned in 0.00 seconds

Table Name: SFP

```

CREATE TABLE SFP (
  SFP_Id NUMBER PRIMARY KEY,
  Student_ID NUMBER,
  form_id NUMBER,
  program_code VARCHAR2(50),
  FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID),
  FOREIGN KEY (form_id) REFERENCES Form(form_id),
  FOREIGN KEY (program_code) REFERENCES Program(program_code)
);
  
```

Oracle Database Express Edition interface showing SQL Commands and Results.

User: SCOTT

SQL Commands:

```

CREATE TABLE SFP (
  SFP_Id NUMBER PRIMARY KEY,
  Student_ID NUMBER,
  form_id NUMBER,
  program_code VARCHAR2(50),
  FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID),
  FOREIGN KEY (form_id) REFERENCES Form(form_id),
  FOREIGN KEY (program_code) REFERENCES Program(program_code)
);

INSERT INTO SFP (SFP_Id, Student_ID, form_id, program_code) VALUES(1, 1, 1, 'CS101')
INSERT INTO SFP (SFP_Id, Student_ID, form_id, program_code) VALUES(2, 2, 2, 'EE102')
INSERT INTO SFP (SFP_Id, Student_ID, form_id, program_code) VALUES(3, 3, 3, 'ME103')
INSERT INTO SFP (SFP_Id, Student_ID, form_id, program_code) VALUES(4, 4, 4, 'BA104')
INSERT INTO SFP (SFP_Id, Student_ID, form_id, program_code) VALUES(5, 5, 5, 'DS105')

select * from SFP
  
```

Results:

SFP_ID	STUDENT_ID	FORM_ID	PROGRAM_CODE
1	1	1	CS101
2	2	2	EE102
3	3	3	ME103
4	4	4	BA104
5	5	5	DS105

5 rows returned in 0.00 seconds

Table Name: Program

```
CREATE TABLE Program (  
    program_code VARCHAR2(50) PRIMARY KEY,  
    program_level VARCHAR2(50),  
    DepPro_Id NUMBER,  
    FOREIGN KEY (DepPro_Id) REFERENCES DepartmentProgram(DepPro_Id)  
);
```

The screenshot shows the Oracle Database Express Edition interface. The SQL Commands window contains the following code:

```
CREATE TABLE Program (  
    program_code VARCHAR2(50) PRIMARY KEY,  
    program_level VARCHAR2(50),  
    DepPro_Id NUMBER,  
    FOREIGN KEY (DepPro_Id) REFERENCES DepartmentProgram(DepPro_Id)  
);  
  
INSERT INTO Program (program_code, program_level, DepPro_Id) VALUES('CS101', 'Undergraduate', 1)  
INSERT INTO Program (program_code, program_level, DepPro_Id) VALUES('EE102', 'Undergraduate', 2)  
INSERT INTO Program (program_code, program_level, DepPro_Id) VALUES('ME103', 'Undergraduate', 3)  
INSERT INTO Program (program_code, program_level, DepPro_Id) VALUES('BA104', 'Postgraduate', 4)  
INSERT INTO Program (program_code, program_level, DepPro_Id) VALUES('DS105', 'Postgraduate', 5)  
  
Select * from Program
```

The Results tab shows the following data:

PROGRAM_CODE	PROGRAM_LEVEL	DEPPRO_ID
CS101	Undergraduate	1
EE102	Undergraduate	2
ME103	Undergraduate	3
BA104	Postgraduate	4
DS105	Postgraduate	5

5 rows returned in 0.00 seconds

Table Name: DepartmentProgram

```
CREATE TABLE DepartmentProgram (  
    DepPro_Id NUMBER PRIMARY KEY,  
    department_name VARCHAR2(100),  
    program_name VARCHAR2(100)  
);
```

The screenshot shows the Oracle Database Express Edition interface. The SQL Commands window contains the following code:

```

CREATE TABLE DepartmentProgram (
  DepPro_Id NUMBER PRIMARY KEY,
  department_name VARCHAR2(100),
  program_name VARCHAR2(100)
);

INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(1, 'Computer Science', 'BSc in Computer Science')
INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(2, 'Electrical Engineering', 'BSc in Electrical Engineering')
INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(3, 'Mechanical Engineering', 'BSc in Mechanical Engineering')
INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(4, 'Business Administration', 'MBA')
INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(5, 'Data Science', 'MSc in CSE')

Select * from DepartmentProgram
  
```

The Results window shows the following data:

DEPPRO_ID	DEPARTMENT_NAME	PROGRAM_NAME
1	Computer Science	BSc in Computer Science
2	Electrical Engineering	BSc in Electrical Engineering
3	Mechanical Engineering	BSc in Mechanical Engineering
4	Business Administration	MBA
5	Data Science	MSc in CSE

5 rows returned in 0.00 seconds

Table Name: ApplicationDocument

```

CREATE TABLE ApplicationDocument (
  AD_Id NUMBER PRIMARY KEY,
  application_id NUMBER,
  document_id NUMBER,
  FOREIGN KEY (application_id) REFERENCES Application(application_id),
  FOREIGN KEY (document_id) REFERENCES Document(document_id)
);
  
```

Table Name: StaffAssignment Table

```

CREATE TABLE StaffAssignment (
  StaffAssignment_Id NUMBER PRIMARY KEY,
  staff_id NUMBER,
  department_id NUMBER,
  FOREIGN KEY (staff_id) REFERENCES Staff(staff_id),
  FOREIGN KEY (department_id) REFERENCES Department(department_id)
);
  
```

Oracle Database Express Edition interface showing SQL commands and results for the StaffAssignment table.

```

CREATE TABLE StaffAssignment (
  StaffAssignment_Id NUMBER PRIMARY KEY,
  staff_id NUMBER,
  department_id NUMBER,
  FOREIGN KEY (staff_id) REFERENCES Staff(staff_id),
  FOREIGN KEY (department_id) REFERENCES Department(department_id)
);
INSERT INTO StaffAssignment (StaffAssignment_Id, staff_id, department_id) VALUES
(5, 5, 5);
Select * from StaffAssignment
  
```

Results:

STAFFASSIGNMENT_ID	STAFF_ID	DEPARTMENT_ID
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5

5 rows returned in 0.00 seconds

Table Name: StaffApplication

```

CREATE TABLE Staff (
  staff_id NUMBER PRIMARY KEY,
  name VARCHAR2(100),
  role VARCHAR2(50),
  staff_department VARCHAR2(100)
);
  
```

Oracle Database Express Edition interface showing SQL commands and results for the Staff table.

```

CREATE TABLE Staff (
  staff_id NUMBER PRIMARY KEY,
  name VARCHAR2(100),
  role VARCHAR2(50),
  staff_department VARCHAR2(100)
);
INSERT INTO Staff (staff_id, name, role, staff_department) VALUES(1, 'Alice Johnson', 'Administrator', 'Admissions')
INSERT INTO Staff (staff_id, name, role, staff_department) VALUES(2, 'Bob Williams', 'Professor', 'Computer Science')
INSERT INTO Staff (staff_id, name, role, staff_department) VALUES(3, 'Charlie Brown', 'Lecturer', 'Electrical Engineering')
INSERT INTO Staff (staff_id, name, role, staff_department) VALUES(4, 'Diana Smith', 'Professor', 'Mechanical Engineering')
INSERT INTO Staff (staff_id, name, role, staff_department) VALUES(5, 'Edward Clark', 'Lecturer', 'Data Science')
select * from Staff
  
```

Results:

STAFF_ID	NAME	ROLE	STAFF_DEPARTMENT
1	Alice Johnson	Administrator	Admissions
2	Bob Williams	Professor	Computer Science
3	Charlie Brown	Lecturer	Electrical Engineering
4	Diana Smith	Professor	Mechanical Engineering
5	Edward Clark	Lecturer	Data Science

5 rows returned in 0.03 seconds

Data Insertion:

Table Name: Test Table

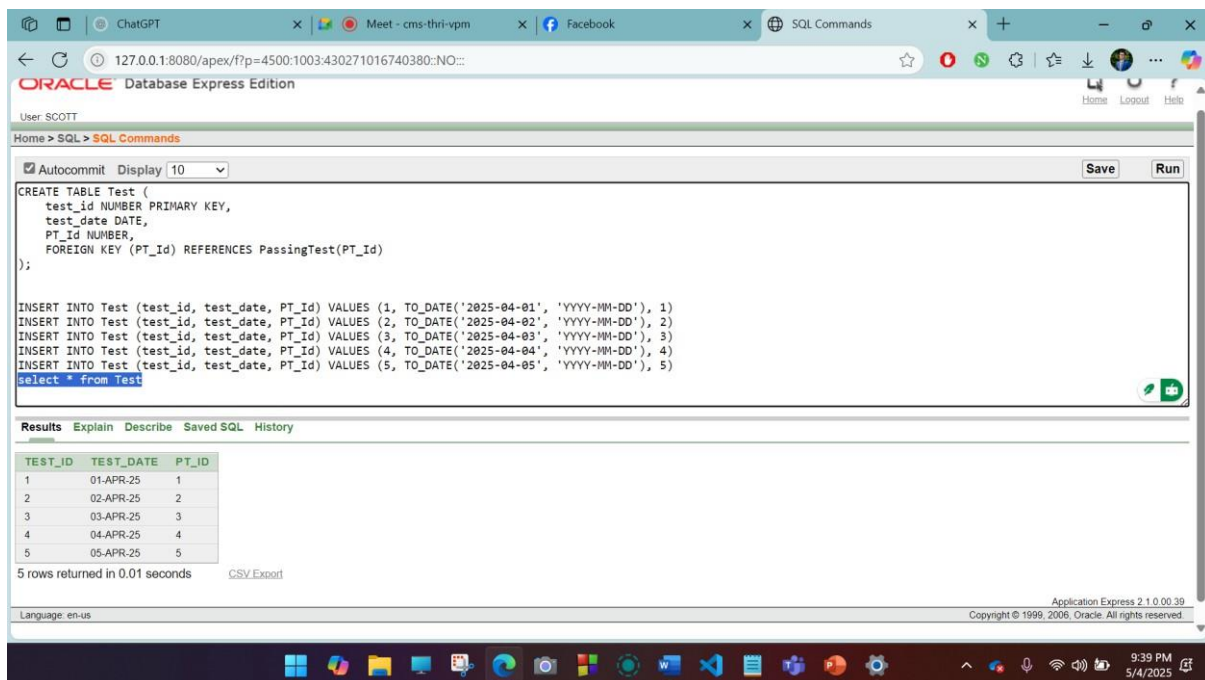
INSERT INTO Test (test_id, test_date, PT_Id) VALUES (1, TO_DATE('2025-04-01', 'YYYY-MM-DD'), 1)

INSERT INTO Test (test_id, test_date, PT_Id) VALUES (2, TO_DATE('2025-04-02', 'YYYY-MM-DD'), 2)

INSERT INTO Test (test_id, test_date, PT_Id) VALUES (3, TO_DATE('2025-04-03', 'YYYY-MM-DD'), 3)

INSERT INTO Test (test_id, test_date, PT_Id) VALUES (4, TO_DATE('2025-04-04', 'YYYY-MM-DD'), 4)

INSERT INTO Test (test_id, test_date, PT_Id) VALUES (5, TO_DATE('2025-04-05', 'YYYY-MM-DD'), 5)



The screenshot shows the Oracle Database Express Edition SQL Commands window. The SQL commands entered are:

```
CREATE TABLE Test (  
  test_id NUMBER PRIMARY KEY,  
  test_date DATE,  
  PT_Id NUMBER,  
  FOREIGN KEY (PT_Id) REFERENCES PassingTest(PT_Id)  
);  
  
INSERT INTO Test (test_id, test_date, PT_Id) VALUES (1, TO_DATE('2025-04-01', 'YYYY-MM-DD'), 1)  
INSERT INTO Test (test_id, test_date, PT_Id) VALUES (2, TO_DATE('2025-04-02', 'YYYY-MM-DD'), 2)  
INSERT INTO Test (test_id, test_date, PT_Id) VALUES (3, TO_DATE('2025-04-03', 'YYYY-MM-DD'), 3)  
INSERT INTO Test (test_id, test_date, PT_Id) VALUES (4, TO_DATE('2025-04-04', 'YYYY-MM-DD'), 4)  
INSERT INTO Test (test_id, test_date, PT_Id) VALUES (5, TO_DATE('2025-04-05', 'YYYY-MM-DD'), 5)  
select * from Test
```

The results of the SQL commands are displayed in a table with 5 rows:

TEST_ID	TEST_DATE	PT_ID
1	01-APR-25	1
2	02-APR-25	2
3	03-APR-25	3
4	04-APR-25	4
5	05-APR-25	5

5 rows returned in 0.01 seconds

Table Name: Passing Test

```
INSERT INTO PassingTest (PT_Id, passing_marks, total_marks) VALUES (1, 50, 100),
INSERT INTO PassingTest (PT_Id, passing_marks, total_marks) VALUES (2, 50, 100),
INSERT INTO PassingTest (PT_Id, passing_marks, total_marks) VALUES (3, 50, 100)
INSERT INTO PassingTest (PT_Id, passing_marks, total_marks) VALUES (4, 50, 100)
INSERT INTO PassingTest (PT_Id, passing_marks, total_marks) VALUES (5, 50, 100)
select * from PassingTest
```

The screenshot shows the Oracle Database Express Edition interface. The SQL Commands window contains the following code:

```
CREATE TABLE PassingTest (
  PT_Id NUMBER PRIMARY KEY,
  passing_marks NUMBER,
  total_marks NUMBER
);

INSERT INTO PassingTest (PT_Id, passing_marks, total_marks) VALUES (1, 50, 100),
INSERT INTO PassingTest (PT_Id, passing_marks, total_marks) VALUES (2, 50, 100),
INSERT INTO PassingTest (PT_Id, passing_marks, total_marks) VALUES (3, 50, 100),
INSERT INTO PassingTest (PT_Id, passing_marks, total_marks) VALUES (4, 50, 100),
INSERT INTO PassingTest (PT_Id, passing_marks, total_marks) VALUES (5, 50, 100)
select * from PassingTest
```

The Results tab shows the output of the SQL statements:

PT_ID	PASSING_MARKS	TOTAL_MARKS
1	50	100
2	50	100
3	50	100
4	50	100
5	50	100

5 rows returned in 0.01 seconds

Application Express 2.1.0.00.39
Copyright © 1999, 2006, Oracle. All rights reserved.

Table Name: DepartmentProgram

```
INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(1, 'Computer Science', 'BSc in Computer Science')
```

```
INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(2, 'Electrical Engineering', 'BSc in Electrical Engineering')
```

```
INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(3, 'Mechanical Engineering', 'BSc in Mechanical Engineering')
```

```
INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(4, 'Business Administration', 'MBA')
```

```
INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(5, 'Data Science', 'MSc in CSE')
```

Select * from DepartmentProgram

The screenshot shows the Oracle Database Express Edition interface. The SQL Commands window contains the following code:

```
CREATE TABLE DepartmentProgram (  
  DepPro_Id NUMBER PRIMARY KEY,  
  department_name VARCHAR2(100),  
  program_name VARCHAR2(100)  
);  
  
INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(1, 'Computer Science', 'BSc in Computer Science')  
INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(2, 'Electrical Engineering', 'BSc in Electrical Engineering')  
INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(3, 'Mechanical Engineering', 'BSc in Mechanical Engineering')  
INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(4, 'Business Administration', 'MBA')  
INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(5, 'Data Science', 'MSc in CSE')
```

Below the code, the query "Select * from DepartmentProgram" is executed. The Results tab shows the following data:

DEPPRO_ID	DEPARTMENT_NAME	PROGRAM_NAME
1	Computer Science	BSc in Computer Science
2	Electrical Engineering	BSc in Electrical Engineering
3	Mechanical Engineering	BSc in Mechanical Engineering
4	Business Administration	MBA
5	Data Science	MSc in CSE

5 rows returned in 0.00 seconds

Table Name: StudentTest

```
INSERT INTO StudentTest (ST_Id, student_id, test_id) VALUES(1, 1, 1)
```

```
INSERT INTO StudentTest (ST_Id, student_id, test_id) VALUES(2, 2, 2)
```

```
INSERT INTO StudentTest (ST_Id, student_id, test_id) VALUES(3, 3, 3)
```

```
INSERT INTO StudentTest (ST_Id, student_id, test_id) VALUES(4, 4, 4)
```

```
INSERT INTO StudentTest (ST_Id, student_id, test_id) VALUES(5, 5, 5)
```

```
select * from StudentTest
```

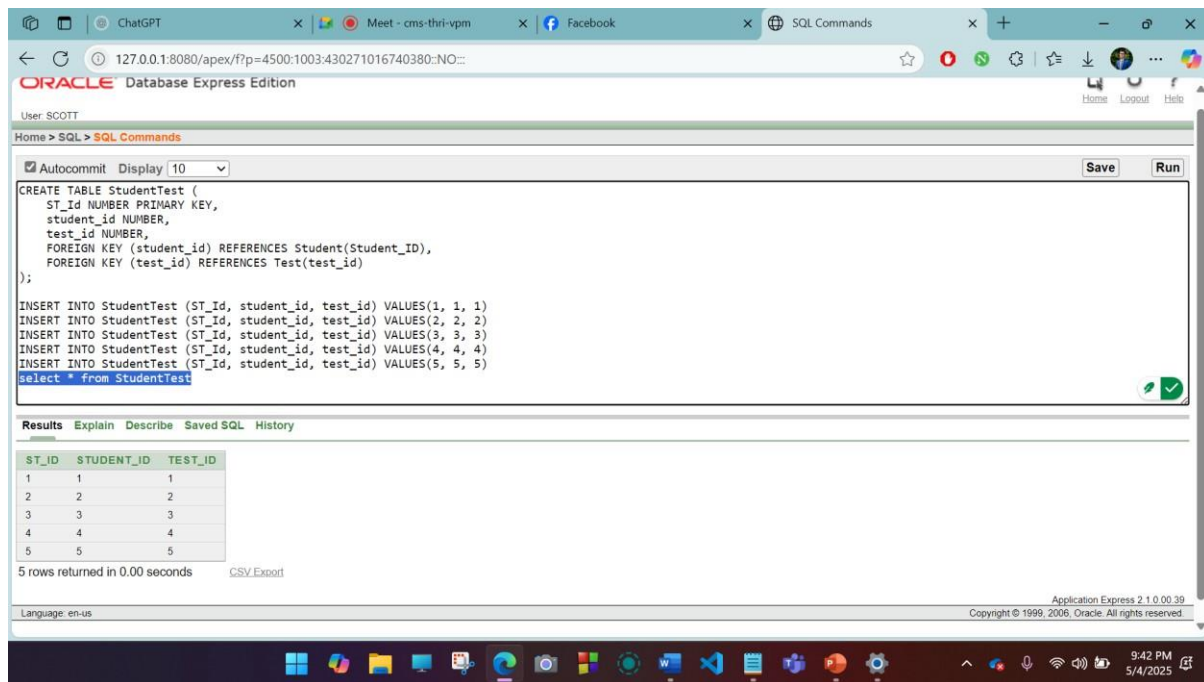


Table Name: Application

INSERT INTO Application (application_id, result_status, comments) VALUES(1, 'Accepted', 'Congratulations on being accepted!')

INSERT INTO Application (application_id, result_status, comments) VALUES(2, 'Rejected', 'Application did not meet criteria.')

INSERT INTO Application (application_id, result_status, comments) VALUES(3, 'Accepted', 'Congratulations on being accepted!')

INSERT INTO Application (application_id, result_status, comments) VALUES(4, 'Under Review', 'Waiting for final decision.')

INSERT INTO Application (application_id, result_status, comments) VALUES(5, 'Accepted', 'Congratulations on being accepted!')

select * from Application

Oracle Database Express Edition SQL Commands window. The user is SCOTT. The SQL commands executed are:

```

CREATE TABLE Application (
  application_id NUMBER PRIMARY KEY,
  result_status VARCHAR2(50),
  comments VARCHAR2(200)
);

INSERT INTO Application (application_id, result_status, comments) VALUES(1, 'Accepted', 'Congratulations on being accepted!')
INSERT INTO Application (application_id, result_status, comments) VALUES(2, 'Rejected', 'Application did not meet criteria.')
INSERT INTO Application (application_id, result_status, comments) VALUES(3, 'Accepted', 'Congratulations on being accepted!')
INSERT INTO Application (application_id, result_status, comments) VALUES(4, 'Under Review', 'Waiting for final decision.')
INSERT INTO Application (application_id, result_status, comments) VALUES(5, 'Accepted', 'Congratulations on being accepted!')
select * from Application
  
```

The results show 5 rows returned in 0.00 seconds:

APPLICATION_ID	RESULT_STATUS	COMMENTS
1	Accepted	Congratulations on being accepted!
2	Rejected	Application did not meet criteria.
3	Accepted	Congratulations on being accepted!
4	Under Review	Waiting for final decision.
5	Accepted	Congratulations on being accepted!

Table Name: Staff

```

INSERT INTO Staff (staff_id, name, role, staff_department) VALUES(1, 'Alice Johnson',
'Administrator', 'Admissions'),
INSERT INTO Staff (staff_id, name, role, staff_department) VALUES
(2, 'Bob Williams', 'Professor', 'Computer Science')
INSERT INTO Staff (staff_id, name, role, staff_department) VALUES
(3, 'Charlie Brown', 'Lecturer', 'Electrical Engineering'),
INSERT INTO Staff (staff_id, name, role, staff_department) VALUES
(4, 'Diana Smith', 'Professor', 'Mechanical Engineering'),
INSERT INTO Staff (staff_id, name, role, staff_department) VALUES
INSERT INTO Staff (staff_id, name, role, staff_department) VALUES
(5, 'Edward Clark', 'Lecturer', 'Data Science')
  
```

Oracle Database Express Edition SQL Commands window. The user is SCOTT. The SQL commands executed are:

```

CREATE TABLE Staff (
  staff_id NUMBER PRIMARY KEY,
  name VARCHAR2(100),
  role VARCHAR2(50),
  staff_department VARCHAR2(100)
);

INSERT INTO Staff (staff_id, name, role, staff_department) VALUES(1, 'Alice Johnson', 'Administrator', 'Admissions')
INSERT INTO Staff (staff_id, name, role, staff_department) VALUES(2, 'Bob Williams', 'Professor', 'Computer Science')
INSERT INTO Staff (staff_id, name, role, staff_department) VALUES(3, 'Charlie Brown', 'Lecturer', 'Electrical Engineering')
INSERT INTO Staff (staff_id, name, role, staff_department) VALUES(4, 'Diana Smith', 'Professor', 'Mechanical Engineering')
INSERT INTO Staff (staff_id, name, role, staff_department) VALUES(5, 'Edward Clark', 'Lecturer', 'Data Science')
select * from Staff
  
```

The results show 5 rows returned in 0.03 seconds:

STAFF_ID	NAME	ROLE	STAFF_DEPARTMENT
1	Alice Johnson	Administrator	Admissions
2	Bob Williams	Professor	Computer Science
3	Charlie Brown	Lecturer	Electrical Engineering
4	Diana Smith	Professor	Mechanical Engineering
5	Edward Clark	Lecturer	Data Science

Table Name: ApplicationDocument

```
INSERT INTO ApplicationDocument (AD_Id, application_id, document_id) VALUES(1, 1, 1),
INSERT INTO ApplicationDocument (AD_Id, application_id, document_id) VALUES
(2, 2, 2),
INSERT INTO ApplicationDocument (AD_Id, application_id, document_id) VALUES
(3, 3, 3),
INSERT INTO ApplicationDocument (AD_Id, application_id, document_id) VALUES
(4, 4, 4),
INSERT INTO ApplicationDocument (AD_Id, application_id, document_id) VALUES
(5, 5, 5);
```

The screenshot shows the Oracle Database Express Edition interface. The SQL Command window contains the following code:

```
CREATE TABLE ApplicationDocument (
  AD_Id NUMBER PRIMARY KEY,
  application_id NUMBER,
  document_id NUMBER,
  FOREIGN KEY (application_id) REFERENCES Application(application_id),
  FOREIGN KEY (document_id) REFERENCES Document(document_id)
);

INSERT INTO ApplicationDocument (AD_Id, application_id, document_id) VALUES(1, 1, 1)
INSERT INTO ApplicationDocument (AD_Id, application_id, document_id) VALUES(2, 2, 2)
INSERT INTO ApplicationDocument (AD_Id, application_id, document_id) VALUES(3, 3, 3)
INSERT INTO ApplicationDocument (AD_Id, application_id, document_id) VALUES(4, 4, 4)
INSERT INTO ApplicationDocument (AD_Id, application_id, document_id) VALUES(5, 5, 5)
select * from ApplicationDocument
```

The Results window displays the following data:

AD_ID	APPLICATION_ID	DOCUMENT_ID
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5

5 rows returned in 0.01 seconds

Table Name: Document

```
INSERT INTO Document (document_id, transcripts, certificates) VALUES(1, 'Transcript 1', 'Certificate 1')
INSERT INTO Document (document_id, transcripts, certificates) VALUES(2, 'Transcript 2', 'Certificate 2')
INSERT INTO Document (document_id, transcripts, certificates) VALUES
(3, 'Transcript 3', 'Certificate 3')
INSERT INTO Document (document_id, transcripts, certificates) VALUES
(4, 'Transcript 4', 'Certificate 4')
INSERT INTO Document (document_id, transcripts, certificates) VALUES
(5, 'Transcript 5', 'Certificate 5')
```

The screenshot shows the Oracle Database Express Edition interface. The SQL Commands window contains the following code:

```
CREATE TABLE Document (  
  document_id NUMBER PRIMARY KEY,  
  transcripts VARCHAR2(200),  
  certificates VARCHAR2(200)  
);  
  
INSERT INTO Document (document_id, transcripts, certificates) VALUES(1, 'Transcript 1', 'Certificate 1')  
INSERT INTO Document (document_id, transcripts, certificates) VALUES(2, 'Transcript 2', 'Certificate 2')  
INSERT INTO Document (document_id, transcripts, certificates) VALUES(3, 'Transcript 3', 'Certificate 3')  
INSERT INTO Document (document_id, transcripts, certificates) VALUES(4, 'Transcript 4', 'Certificate 4')  
INSERT INTO Document (document_id, transcripts, certificates) VALUES(5, 'Transcript 5', 'Certificate 5')  
  
select * from Document
```

The Results tab shows the following data:

DOCUMENT_ID	TRANSCRIPTS	CERTIFICATES
1	Transcript 1	Certificate 1
2	Transcript 2	Certificate 2
3	Transcript 3	Certificate 3
4	Transcript 4	Certificate 4
5	Transcript 5	Certificate 5

5 rows returned in 0.00 seconds

Table Name: ApplicationStaff

INSERT INTO ApplicationStaff (AS_Id, application_id, staff_id) VALUES(1, 1, 1)

INSERT INTO ApplicationStaff (AS_Id, application_id, staff_id) VALUES
(2, 2, 2)

INSERT INTO ApplicationStaff (AS_Id, application_id, staff_id) VALUES
(3, 3, 3)

(4, 4, 4)

INSERT INTO ApplicationStaff (AS_Id, application_id, staff_id) VALUES
(5, 5, 5)

The screenshot shows the Oracle Database Express Edition interface. The SQL Command window contains the following code:

```
CREATE TABLE ApplicationStaff (  
  AS_Id NUMBER PRIMARY KEY,  
  application_id NUMBER,  
  staff_id NUMBER,  
  FOREIGN KEY (application_id) REFERENCES Application(application_id),  
  FOREIGN KEY (staff_id) REFERENCES Staff(staff_id)  
);  
  
INSERT INTO ApplicationStaff (AS_Id, application_id, staff_id) VALUES(1, 1, 1)  
INSERT INTO ApplicationStaff (AS_Id, application_id, staff_id) VALUES(2, 2, 2)  
INSERT INTO ApplicationStaff (AS_Id, application_id, staff_id) VALUES(3, 3, 3)  
INSERT INTO ApplicationStaff (AS_Id, application_id, staff_id) VALUES(4, 4, 4)  
INSERT INTO ApplicationStaff (AS_Id, application_id, staff_id) VALUES(5, 5, 5)  
select * from ApplicationStaff
```

The Results window shows the following data:

AS_ID	APPLICATION_ID	STAFF_ID
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5

5 rows returned in 0.00 seconds

Table Name: ApplicationTest

INSERT INTO ApplicationTest (AT_Id, application_id, test_id) VALUES(1, 1, 1)

INSERT INTO ApplicationTest (AT_Id, application_id, test_id) VALUES
(2, 2, 2)

INSERT INTO ApplicationTest (AT_Id, application_id, test_id) VALUES
(3, 3, 3)

INSERT INTO ApplicationTest (AT_Id, application_id, test_id) VALUES
(4, 4, 4)

INSERT INTO ApplicationTest (AT_Id, application_id, test_id) VALUES
(5, 5, 5)

Oracle Database Express Edition interface showing SQL commands and results for the ApplicationTest table.

```

CREATE TABLE ApplicationTest (
  AT_Id NUMBER PRIMARY KEY,
  application_id NUMBER,
  test_id NUMBER,
  FOREIGN KEY (application_id) REFERENCES Application(application_id),
  FOREIGN KEY (test_id) REFERENCES Test(test_id)
);

INSERT INTO ApplicationTest (AT_Id, application_id, test_id) VALUES(1, 1, 1)
INSERT INTO ApplicationTest (AT_Id, application_id, test_id) VALUES(2, 2, 2)
INSERT INTO ApplicationTest (AT_Id, application_id, test_id) VALUES(3, 3, 3)
INSERT INTO ApplicationTest (AT_Id, application_id, test_id) VALUES(4, 4, 4)
INSERT INTO ApplicationTest (AT_Id, application_id, test_id) VALUES(5, 5, 5)
select * from ApplicationTest
  
```

AT_ID	APPLICATION_ID	TEST_ID
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5

5 rows returned in 0.02 seconds

Table Name: StaffAssignment Table

INSERT INTO StaffAssignment (StaffAssignment_Id, staff_id, department_id) VALUES

(1, 1, 1)

INSERT INTO StaffAssignment (StaffAssignment_Id, staff_id, department_id) VALUES

(2, 2, 2)

INSERT INTO StaffAssignment (StaffAssignment_Id, staff_id, department_id) VALUES

(3, 3, 3)

INSERT INTO StaffAssignment (StaffAssignment_Id, staff_id, department_id) VALUES

(4, 4, 4)

INSERT INTO StaffAssignment (StaffAssignment_Id, staff_id, department_id) VALUES

(5, 5, 5);

Oracle Database Express Edition interface showing SQL commands and results for the StaffAssignment table.

```

CREATE TABLE StaffAssignment (
  StaffAssignment_Id NUMBER PRIMARY KEY,
  staff_id NUMBER,
  department_id NUMBER,
  FOREIGN KEY (staff_id) REFERENCES Staff(staff_id),
  FOREIGN KEY (department_id) REFERENCES Department(department_id)
);

INSERT INTO StaffAssignment (StaffAssignment_Id, staff_id, department_id) VALUES
(5, 5, 5);

Select * from StaffAssignment
  
```

STAFFASSIGNMENT_ID	STAFF_ID	DEPARTMENT_ID
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5

5 rows returned in 0.00 seconds

Table Name: Form

```
INSERT INTO Form (form_id, date_of_submission, selected_program, uploaded_documents) VALUES
(1, TO_DATE('2025-04-01', 'YYYY-MM-DD'), 'Computer Science', 'Transcript, CV')
INSERT INTO Form (form_id, date_of_submission, selected_program, uploaded_documents) VALUES
(2, TO_DATE('2025-04-02', 'YYYY-MM-DD'), 'Electrical Engineering', 'Transcript, Recommendation Letter')
INSERT INTO Form (form_id, date_of_submission, selected_program, uploaded_documents) VALUES
(3, TO_DATE('2025-04-03', 'YYYY-MM-DD'), 'Mechanical Engineering', 'Transcript, Portfolio')
INSERT INTO Form (form_id, date_of_submission, selected_program, uploaded_documents) VALUES
(4, TO_DATE('2025-04-04', 'YYYY-MM-DD'), 'Business Administration', 'Transcript, Personal Statement')
INSERT INTO Form (form_id, date_of_submission, selected_program, uploaded_documents) VALUES
(5, TO_DATE('2025-04-05', 'YYYY-MM-DD'), 'Data Science', 'Transcript, Research Paper')
```

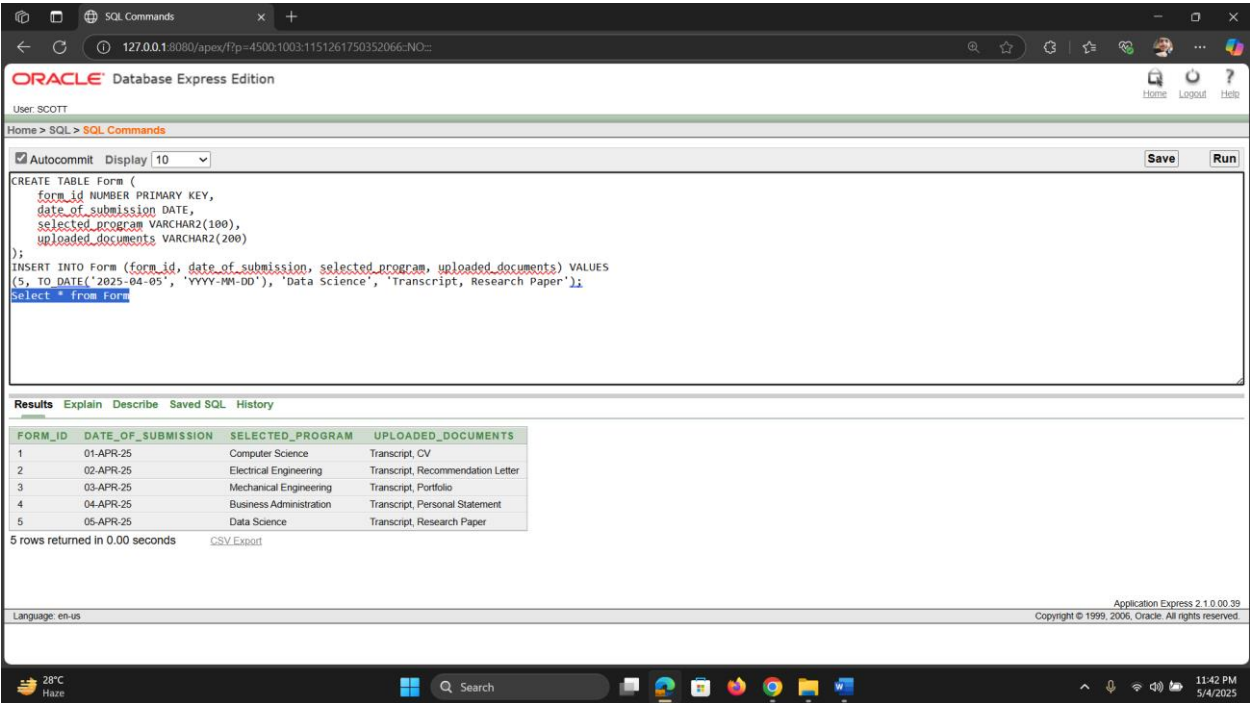


Table Name: Student

```
INSERT INTO Student (Student_ID, Name, Phone, Email, City, Street_No, House_No) VALUES
(1, 'John Doe', '1234567890', 'john.doe@example.com', 'Dhaka', 'Street 1', 'House 101')
INSERT INTO Student (Student_ID, Name, Phone, Email, City, Street_No, House_No) VALUES
(2, 'Jane Smith', '0987654321', 'jane.smith@example.com', 'Chittagong', 'Street 2', 'House 102')
INSERT INTO Student (Student_ID, Name, Phone, Email, City, Street_No, House_No) VALUES
(3, 'Ali Khan', '1234123412', 'ali.khan@example.com', 'Sylhet', 'Street 3', 'House 103')
INSERT INTO Student (Student_ID, Name, Phone, Email, City, Street_No, House_No) VALUES
(4, 'Sara Ali', '2345234523', 'sara.ali@example.com', 'Rajshahi', 'Street 4', 'House 104')
INSERT INTO Student (Student_ID, Name, Phone, Email, City, Street_No, House_No) VALUES
(5, 'Mehdi Hasan', '3456345634', 'mehdi.hasan@example.com', 'Barisal', 'Street 5', 'House 105')
```

The screenshot shows the Oracle Database Express Edition SQL Commands window. The user is SCOTT. The SQL Commands tab is active, showing the following code:

```

CREATE TABLE Student (
  Student_ID NUMBER PRIMARY KEY,
  Name VARCHAR2(100),
  Phone VARCHAR2(15),
  Email VARCHAR2(100),
  City VARCHAR2(50),
  Street_No VARCHAR2(20),
  House_No VARCHAR2(20)
);
INSERT INTO Student (Student_ID, Name, Phone, Email, City, Street_No, House_No) VALUES
(5, 'Mehdi Hasan', '3456345634', 'mehdi.hasan@example.com', 'Barisal', 'Street 5', 'House 105')

Select * from Student
  
```

The Results tab is active, showing the following table:

STUDENT_ID	NAME	PHONE	EMAIL	CITY	STREET_NO	HOUSE_NO
1	John Doe	1234567890	john.doe@example.com	Dhaka	Street 1	House 101
2	Jane Smith	0987654321	jane.smith@example.com	Chittagong	Street 2	House 102
3	Ali Khan	1234123412	ali.khan@example.com	Sylhet	Street 3	House 103
4	Sara Ali	2345234523	sara.ali@example.com	Rajshahi	Street 4	House 104
5	Mehdi Hasan	3456345634	mehdi.hasan@example.com	Barisal	Street 5	House 105

5 rows returned in 0.00 seconds

Table Name: Department Table

INSERT INTO Department (department_id, department_name) VALUES

(1, 'Computer Science')

INSERT INTO Department (department_id, department_name) VALUES

(2, 'Electrical Engineering')

INSERT INTO Department (department_id, department_name) VALUES

(3, 'Mechanical Engineering')

INSERT INTO Department (department_id, department_name) VALUES

(4, 'Business Administration')

INSERT INTO Department (department_id, department_name) VALUES

(5, 'Data Science')

The screenshot shows the Oracle Database Express Edition SQL Commands window. The user is SCOTT. The SQL Commands tab is active, showing the following code:

```

CREATE TABLE Department (
  department_id NUMBER PRIMARY KEY,
  department_name VARCHAR2(100)
);
INSERT INTO Department (department_id, department_name) VALUES
(5, 'Data Science');

Select * from Department
  
```

The Results tab is active, showing the following table:

DEPARTMENT_ID	DEPARTMENT_NAME
1	Computer Science
2	Electrical Engineering
3	Mechanical Engineering
4	Business Administration
5	Data Science

5 rows returned in 0.00 seconds

Table Name: DepPro_Assignment

INSERT INTO DepPro_Assignment (DP_Id, program_code, department_id) VALUES(1, 'CS101', 1)

INSERT INTO DepPro_Assignment (DP_Id, program_code, department_id) VALUES(2, 'EE102', 2)

INSERT INTO DepPro_Assignment (DP_Id, program_code, department_id) VALUES(3, 'ME103', 3)

INSERT INTO DepPro_Assignment (DP_Id, program_code, department_id) VALUES(4, 'BA104', 4)

INSERT INTO DepPro_Assignment (DP_Id, program_code, department_id) VALUES(5, 'DS105', 5)

The screenshot shows the Oracle Database Express Edition interface. The SQL Commands window contains the following code:

```
CREATE TABLE DepPro_Assignment (  
  DP_Id NUMBER PRIMARY KEY,  
  program_code VARCHAR2(50),  
  department_id NUMBER,  
  FOREIGN KEY (program_code) REFERENCES Program(program_code),  
  FOREIGN KEY (department_id) REFERENCES Department(department_id)  
);  
  
INSERT INTO DepPro_Assignment (DP_Id, program_code, department_id) VALUES(1, 'CS101', 1)  
INSERT INTO DepPro_Assignment (DP_Id, program_code, department_id) VALUES(2, 'EE102', 2)  
INSERT INTO DepPro_Assignment (DP_Id, program_code, department_id) VALUES(3, 'ME103', 3)  
INSERT INTO DepPro_Assignment (DP_Id, program_code, department_id) VALUES(4, 'BA104', 4)  
INSERT INTO DepPro_Assignment (DP_Id, program_code, department_id) VALUES(5, 'DS105', 5)
```

Below the code, the 'Results' tab is selected, displaying the following data:

DP_ID	PROGRAM_CODE	DEPARTMENT_ID
1	CS101	1
2	EE102	2
3	ME103	3
4	BA104	4
5	DS105	5

5 rows returned in 0.00 seconds

Table Name: SFP

INSERT INTO SFP (SFP_Id, Student_ID, form_id, program_code) VALUES(1, 1, 1, 'CS101')

INSERT INTO SFP (SFP_Id, Student_ID, form_id, program_code) VALUES(2, 2, 2, 'EE102')

INSERT INTO SFP (SFP_Id, Student_ID, form_id, program_code) VALUES(3, 3, 3, 'ME103')

INSERT INTO SFP (SFP_Id, Student_ID, form_id, program_code) VALUES(4, 4, 4, 'BA104')

INSERT INTO SFP (SFP_Id, Student_ID, form_id, program_code) VALUES(5, 5, 5, 'DS105')

select * from SFP

Oracle Database Express Edition interface showing SQL commands and results.

User: SCOTT

Home > SQL > SQL Commands

Autocommit Display 10

Save Run

```

CREATE TABLE SFP (
  SFP_Id NUMBER PRIMARY KEY,
  Student_ID NUMBER,
  form_id NUMBER,
  program_code VARCHAR2(50),
  FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID),
  FOREIGN KEY (form_id) REFERENCES Form(form_id),
  FOREIGN KEY (program_code) REFERENCES Program(program_code)
);

INSERT INTO SFP (SFP_Id, Student_ID, form_id, program_code) VALUES(1, 1, 1, 'CS101')
INSERT INTO SFP (SFP_Id, Student_ID, form_id, program_code) VALUES(2, 2, 2, 'EE102')
INSERT INTO SFP (SFP_Id, Student_ID, form_id, program_code) VALUES(3, 3, 3, 'ME103')
INSERT INTO SFP (SFP_Id, Student_ID, form_id, program_code) VALUES(4, 4, 4, 'BA104')
INSERT INTO SFP (SFP_Id, Student_ID, form_id, program_code) VALUES(5, 5, 5, 'DS105')

select * from SFP
  
```

Results Explain Describe Saved SQL History

SFP_ID	STUDENT_ID	FORM_ID	PROGRAM_CODE
1	1	1	CS101
2	2	2	EE102
3	3	3	ME103
4	4	4	BA104
5	5	5	DS105

5 rows returned in 0.00 seconds CSV Export

Table Name: Program

```

INSERT INTO Program (program_code, program_level, DepPro_Id) VALUES('CS101', 'Undergraduate', 1)
INSERT INTO Program (program_code, program_level, DepPro_Id) VALUES('EE102', 'Undergraduate', 2)
INSERT INTO Program (program_code, program_level, DepPro_Id) VALUES('ME103', 'Undergraduate', 3)
INSERT INTO Program (program_code, program_level, DepPro_Id) VALUES('BA104', 'Postgraduate', 4)
INSERT INTO Program (program_code, program_level, DepPro_Id) VALUES('DS105', 'Postgraduate', 5)
  
```

Select * from Program

Oracle Database Express Edition interface showing SQL commands and results.

User: SCOTT

Home > SQL > SQL Commands

Autocommit Display 10

Save Run

```

CREATE TABLE Program (
  program_code VARCHAR2(50) PRIMARY KEY,
  program_level VARCHAR2(50),
  DepPro_Id NUMBER,
  FOREIGN KEY (DepPro_Id) REFERENCES DepartmentProgram(DepPro_Id)
);

INSERT INTO Program (program_code, program_level, DepPro_Id) VALUES('CS101', 'Undergraduate', 1)
INSERT INTO Program (program_code, program_level, DepPro_Id) VALUES('EE102', 'Undergraduate', 2)
INSERT INTO Program (program_code, program_level, DepPro_Id) VALUES('ME103', 'Undergraduate', 3)
INSERT INTO Program (program_code, program_level, DepPro_Id) VALUES('BA104', 'Postgraduate', 4)
INSERT INTO Program (program_code, program_level, DepPro_Id) VALUES('DS105', 'Postgraduate', 5)

Select * from Program
  
```

Results Explain Describe Saved SQL History

PROGRAM_CODE	PROGRAM_LEVEL	DEPPRO_ID
CS101	Undergraduate	1
EE102	Undergraduate	2
ME103	Undergraduate	3
BA104	Postgraduate	4
DS105	Postgraduate	5

5 rows returned in 0.00 seconds CSV Export

Table Name: DepartmentProgram

INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(1, 'Computer Science', 'BSc in Computer Science')

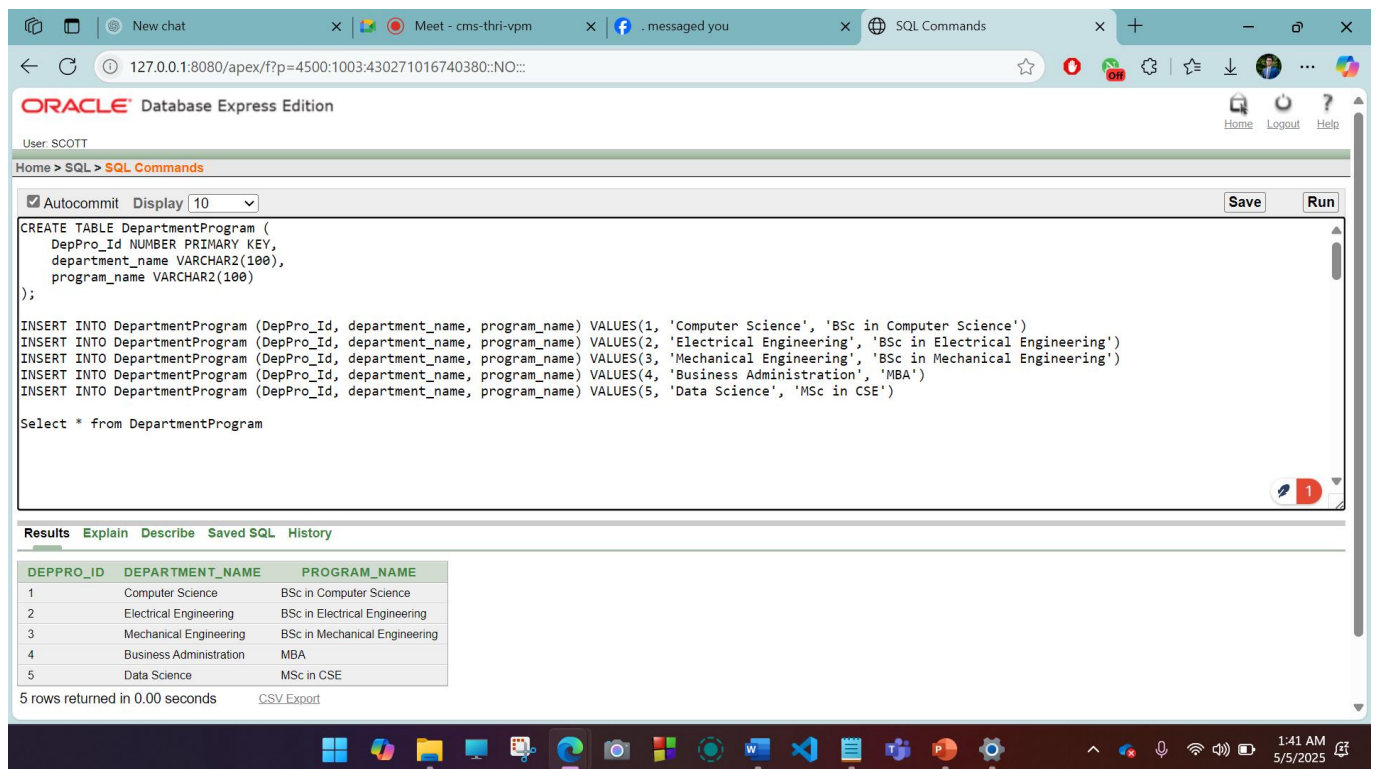
INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(2, 'Electrical Engineering', 'BSc in Electrical Engineering')

INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(3, 'Mechanical Engineering', 'BSc in Mechanical Engineering')

INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(4, 'Business Administration', 'MBA')

INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(5, 'Data Science', 'MSc in CSE')

Select * from DepartmentProgram



The screenshot shows the Oracle Database Express Edition SQL Commands window. The SQL script executed is as follows:

```
CREATE TABLE DepartmentProgram (  
  DepPro_Id NUMBER PRIMARY KEY,  
  department_name VARCHAR2(100),  
  program_name VARCHAR2(100)  
);  
  
INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(1, 'Computer Science', 'BSc in Computer Science')  
INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(2, 'Electrical Engineering', 'BSc in Electrical Engineering')  
INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(3, 'Mechanical Engineering', 'BSc in Mechanical Engineering')  
INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(4, 'Business Administration', 'MBA')  
INSERT INTO DepartmentProgram (DepPro_Id, department_name, program_name) VALUES(5, 'Data Science', 'MSc in CSE')  
  
Select * from DepartmentProgram
```

The query results are displayed in a table with 5 rows:

DEPPRO_ID	DEPARTMENT_NAME	PROGRAM_NAME
1	Computer Science	BSc in Computer Science
2	Electrical Engineering	BSc in Electrical Engineering
3	Mechanical Engineering	BSc in Mechanical Engineering
4	Business Administration	MBA
5	Data Science	MSc in CSE

5 rows returned in 0.00 seconds

Table Name: ApplicationDocument

INSERT INTO ApplicationDocument (AD_Id, application_id, document_id) VALUES(1, 1, 1),

INSERT INTO ApplicationDocument (AD_Id, application_id, document_id) VALUES
(2, 2, 2),

INSERT INTO ApplicationDocument (AD_Id, application_id, document_id) VALUES
(3, 3, 3),

INSERT INTO ApplicationDocument (AD_Id, application_id, document_id) VALUES

(4, 4, 4),

INSERT INTO ApplicationDocument (AD_Id, application_id, document_id) VALUES

(5, 5, 5);

Table Name: StaffAssignment Table

INSERT INTO StaffAssignment (StaffAssignment_Id, staff_id, department_id) VALUES

(1, 1, 1)

INSERT INTO StaffAssignment (StaffAssignment_Id, staff_id, department_id) VALUES

(2, 2, 2)

INSERT INTO StaffAssignment (StaffAssignment_Id, staff_id, department_id) VALUES

(3, 3, 3)

INSERT INTO StaffAssignment (StaffAssignment_Id, staff_id, department_id) VALUES

(4, 4, 4)

INSERT INTO StaffAssignment (StaffAssignment_Id, staff_id, department_id) VALUES

(5, 5, 5);

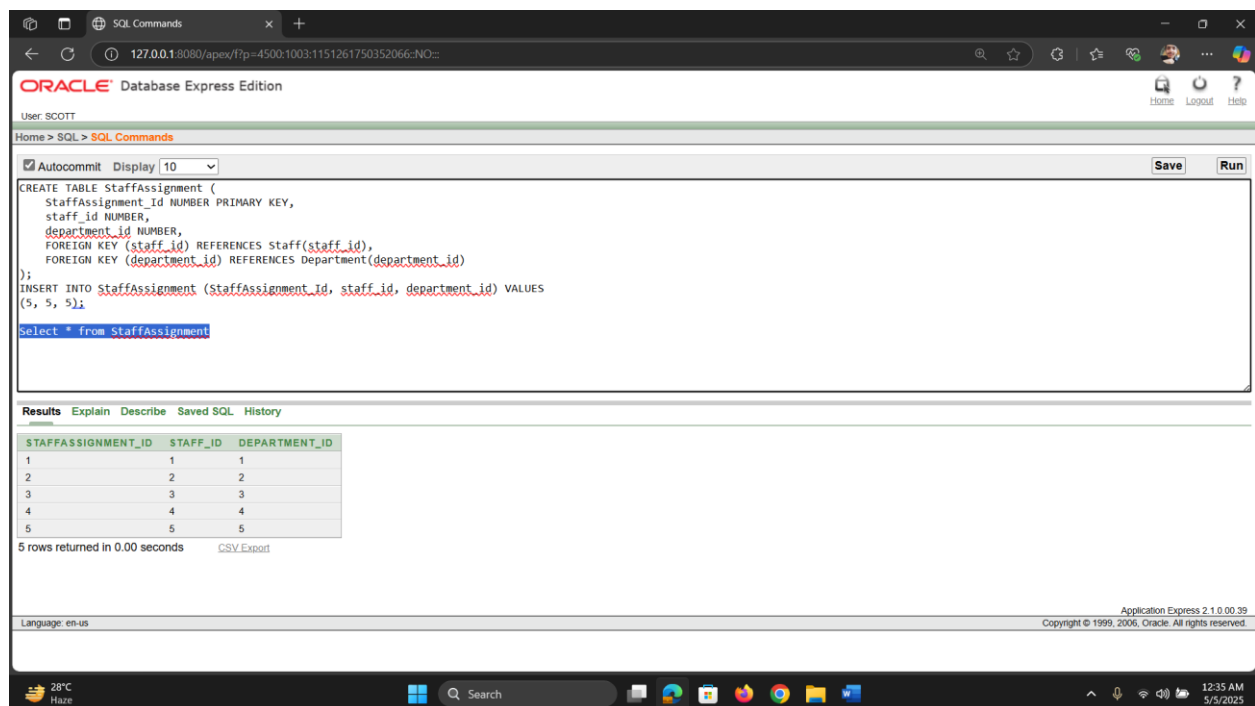


Table Name: StaffApplication

INSERT INTO Staff (staff_id, name, role, staff_department) VALUES(1, 'Alice Johnson', 'Administrator', 'Admissions'),

INSERT INTO Staff (staff_id, name, role, staff_department) VALUES
(2, 'Bob Williams', 'Professor', 'Computer Science')

INSERT INTO Staff (staff_id, name, role, staff_department) VALUES
(3, 'Charlie Brown', 'Lecturer', 'Electrical Engineering'),

INSERT INTO Staff (staff_id, name, role, staff_department) VALUES
(4, 'Diana Smith', 'Professor', 'Mechanical Engineering'),

INSERT INTO Staff (staff_id, name, role, staff_department) VALUES

INSERT INTO Staff (staff_id, name, role, staff_department) VALUES

(5, 'Edward Clark', 'Lecturer', 'Data Science')

The screenshot shows the Oracle Database Express Edition interface. The SQL Command window contains the following code:

```

CREATE TABLE Staff (
  staff_id NUMBER PRIMARY KEY,
  name VARCHAR2(100),
  role VARCHAR2(50),
  staff_department VARCHAR2(100)
);

INSERT INTO Staff (staff_id, name, role, staff_department) VALUES(1, 'Alice Johnson', 'Administrator', 'Admissions')
INSERT INTO Staff (staff_id, name, role, staff_department) VALUES(2, 'Bob Williams', 'Professor', 'Computer Science')
INSERT INTO Staff (staff_id, name, role, staff_department) VALUES(3, 'Charlie Brown', 'Lecturer', 'Electrical Engineering')
INSERT INTO Staff (staff_id, name, role, staff_department) VALUES(4, 'Diana Smith', 'Professor', 'Mechanical Engineering')
INSERT INTO Staff (staff_id, name, role, staff_department) VALUES(5, 'Edward Clark', 'Lecturer', 'Data Science')
select * from Staff
  
```

The Results tab shows the following data:

STAFF_ID	NAME	ROLE	STAFF_DEPARTMENT
1	Alice Johnson	Administrator	Admissions
2	Bob Williams	Professor	Computer Science
3	Charlie Brown	Lecturer	Electrical Engineering
4	Diana Smith	Professor	Mechanical Engineering
5	Edward Clark	Lecturer	Data Science

5 rows returned in 0.03 seconds

Basic PL/SQL:

1. Variables

Question 1:

"Retrieve the name and phone number of the student with Student_ID = 1 and store them in variables."

-- Project: University Admission Management System

-- Semester: Spring 2024-2025

-- Course: Advanced Database Management Systems

-- Section: A

DECLARE

 v_student_name VARCHAR2(100);

 v_student_phone VARCHAR2(15);

BEGIN

 SELECT name, phone

 INTO v_student_name, v_student_phone

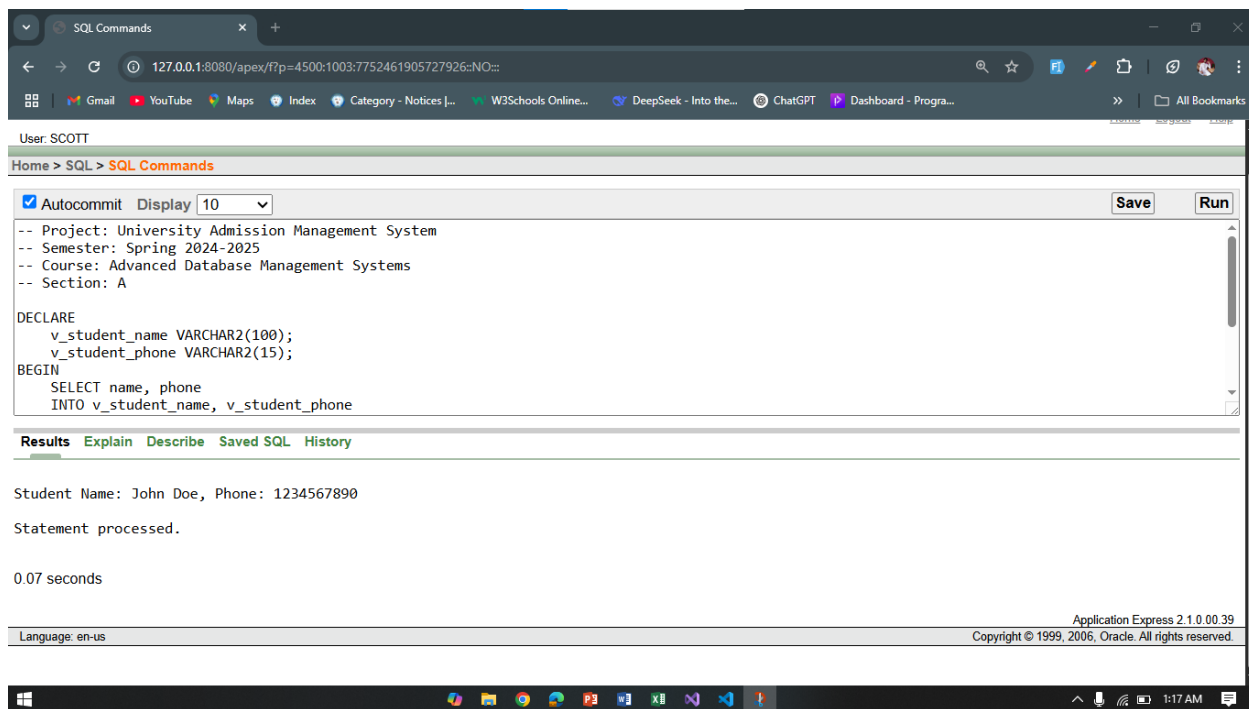
 FROM Student

 WHERE student_id = 1;

 DBMS_OUTPUT.PUT_LINE('Student Name: ' || v_student_name || ', Phone: ' || v_student_phone);

END;

/



Question 2:

"Retrieve and display the Student_ID and Email for students who have result_status = 'Accepted'."

```
-- Project: University Admission Management System
-- Semester: Spring 2024-2025
-- Course: Advanced Database Management Systems
-- Section: A
```

DECLARE

```
v_student_id NUMBER;
v_student_email VARCHAR2(100);
```

BEGIN

```
SELECT student_id, email
INTO v_student_id, v_student_email
FROM Student
```

```
WHERE student_id = (SELECT student_id FROM Application WHERE result_status = 'Accepted'
AND ROWNUM = 1);
```

```
DBMS_OUTPUT.PUT_LINE('Student ID: ' || v_student_id || ', Email: ' || v_student_email);
```

END;

/

2. Operators

Question 1:

"Compare if the student's total_marks are greater than the passing marks, and output if they passed or failed."

```
-- Project: University Admission Management System
-- Semester: Spring 2024-2025
```

-- Course: Advanced Database Management Systems
-- Section: A

DECLARE

```
v_total_marks NUMBER := 75;  
v_passing_marks NUMBER := 50;  
v_result VARCHAR2(10);
```

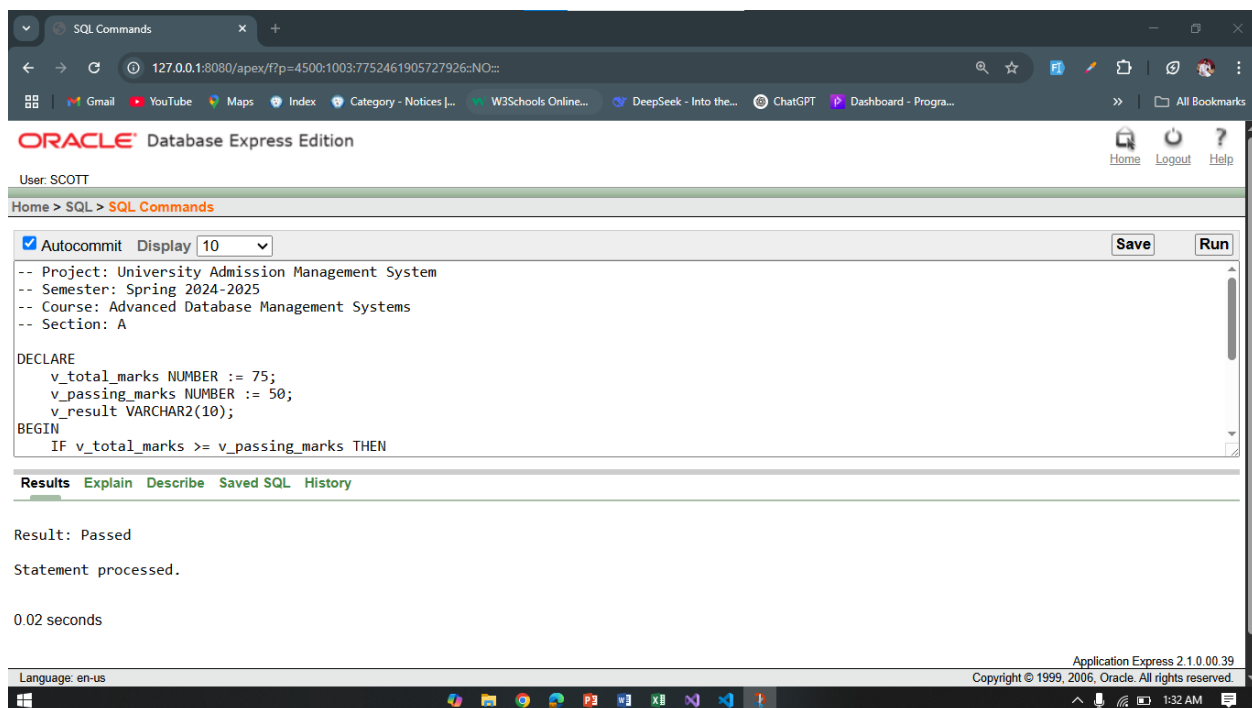
BEGIN

```
IF v_total_marks >= v_passing_marks THEN  
    v_result := 'Passed';  
ELSE  
    v_result := 'Failed';  
END IF;
```

```
DBMS_OUTPUT.PUT_LINE('Result: ' || v_result);
```

END;

/



Question 2:

"Check if a student is in the top 5 based on their score."

-- Project: University Admission Management System
-- Semester: Spring 2024-2025
-- Course: Advanced Database Management Systems
-- Section: A

DECLARE

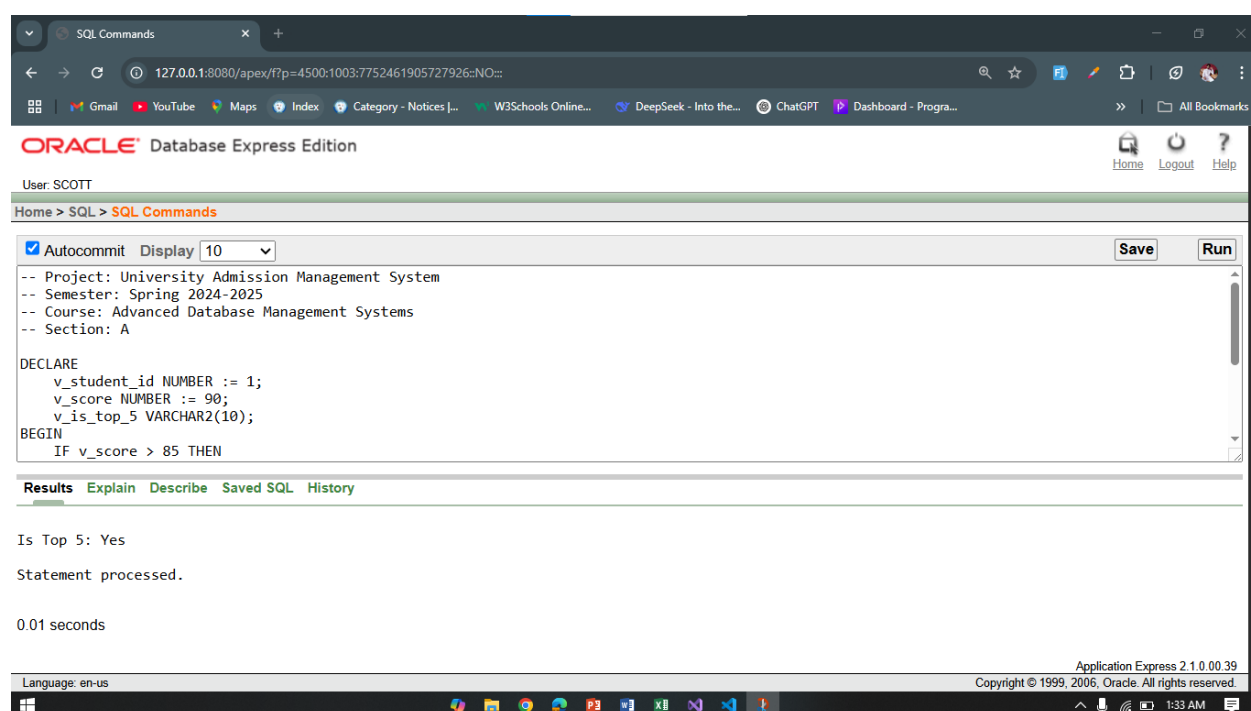
```
v_student_id NUMBER := 1;  
v_score NUMBER := 90;
```

```

v_is_top_5 VARCHAR2(10);
BEGIN
    IF v_score > 85 THEN
        v_is_top_5 := 'Yes';
    ELSE
        v_is_top_5 := 'No';
    END IF;

    DBMS_OUTPUT.PUT_LINE('Is Top 5: ' || v_is_top_5);
END;
/

```



3. Single-Row Functions

Question 1:

"Retrieve the student's name in uppercase."

```

-- Project: University Admission Management System
-- Semester: Spring 2024-2025
-- Course: Advanced Database Management Systems
-- Section: A

```

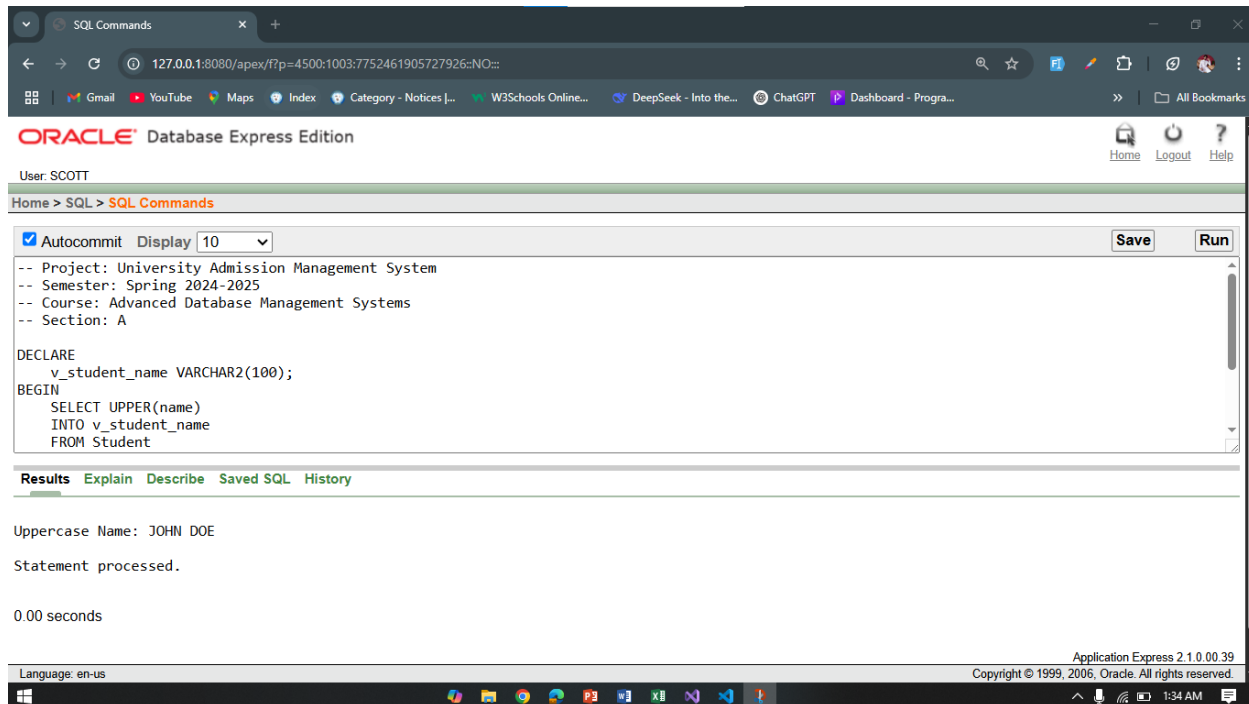
```

DECLARE
    v_student_name VARCHAR2(100);
BEGIN
    SELECT UPPER(name)
    INTO v_student_name
    FROM Student
    WHERE student_id = 1;

```



```
DBMS_OUTPUT.PUT_LINE('Uppercase Name: ' || v_student_name);  
END;  
/
```



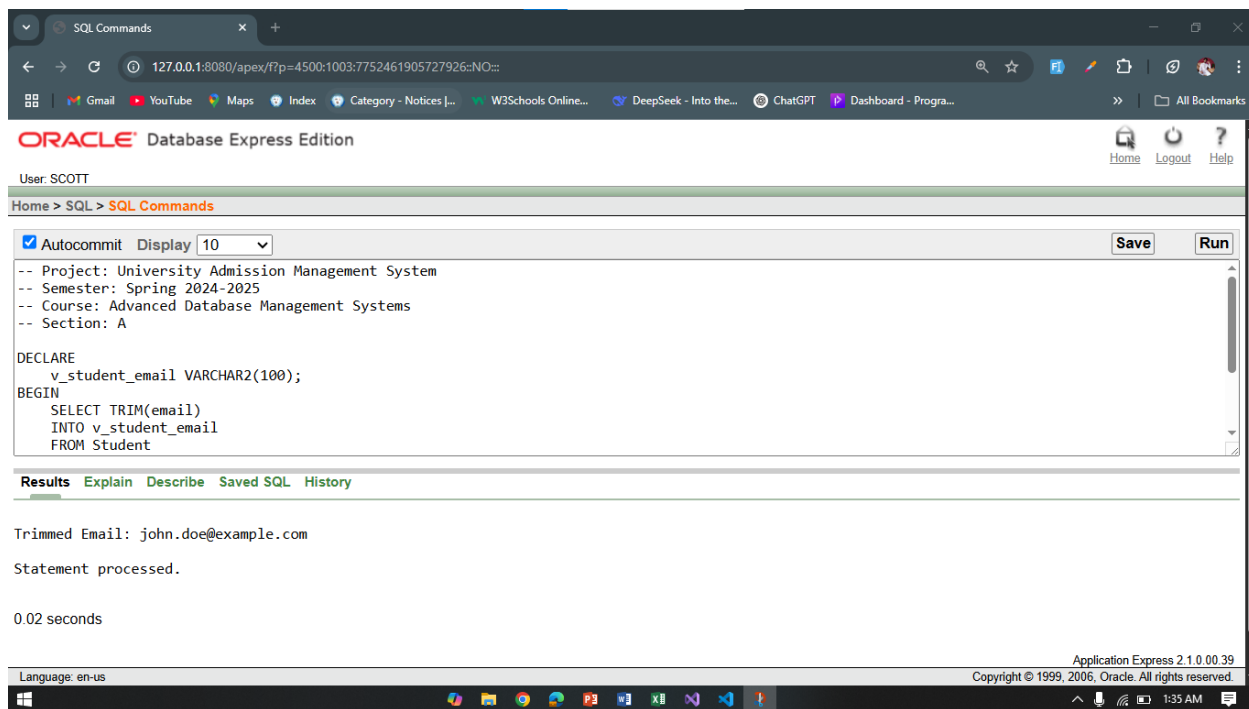
Question 2:

"Get the email of a student and remove leading/trailing spaces."

```
-- Project: University Admission Management System  
-- Semester: Spring 2024-2025  
-- Course: Advanced Database Management Systems  
-- Section: A
```

```
DECLARE  
  v_student_email VARCHAR2(100);  
BEGIN  
  SELECT TRIM(email)  
  INTO v_student_email  
  FROM Student  
  WHERE student_id = 1;
```

```
DBMS_OUTPUT.PUT_LINE('Trimmed Email: ' || v_student_email);  
END;  
/
```

4. Group Functions

Question 1:

"Find the average total_marks of all students."

-- Project: University Admission Management System
-- Semester: Spring 2024-2025
-- Course: Advanced Database Management Systems
-- Section: A

DECLARE

 v_avg_marks NUMBER;

BEGIN

 SELECT AVG(total_marks) INTO v_avg_marks
 FROM Test;

 DBMS_OUTPUT.PUT_LINE('Average Marks: ' || v_avg_marks);

END;

/

Question 2:

"Find the total number of students who have `result\_status = 'Accepted'."`

-- Project: University Admission Management System
-- Semester: Spring 2024-2025
-- Course: Advanced Database Management Systems
-- Section: A

DECLARE

 v_total_accepted NUMBER;

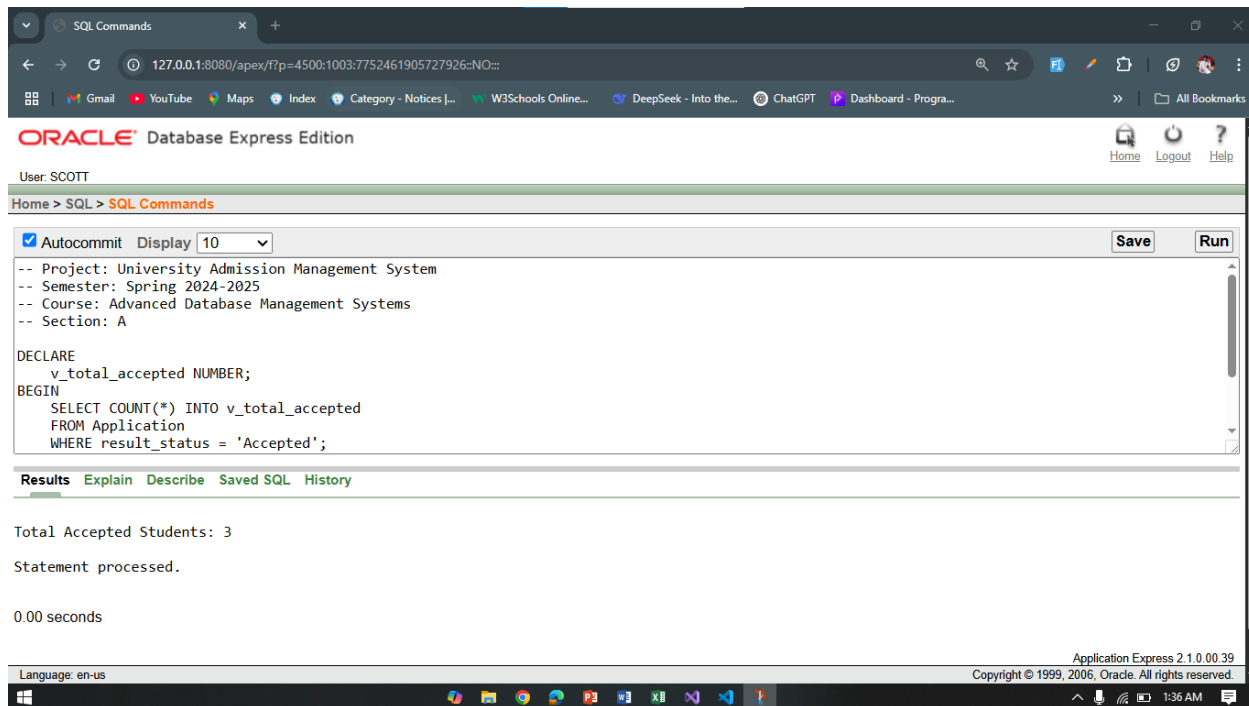
BEGIN

```

SELECT COUNT(*) INTO v_total_accepted
FROM Application
WHERE result_status = 'Accepted';

DBMS_OUTPUT.PUT_LINE('Total Accepted Students: ' || v_total_accepted);
END;
/

```



5. Loop

Question 1:

"Iterate over the first 5 students and print their names."

```

-- Project: University Admission Management System
-- Semester: Spring 2024-2025
-- Course: Advanced Database Management Systems
-- Section: A

```

```

BEGIN

```

```

  FOR i IN 1..5 LOOP
    DBMS_OUTPUT.PUT_LINE('Student ' || i || ' Name: ' || (SELECT name FROM Student WHERE
ROWNUM = i));
  END LOOP;
END;
/

```

Question 2:

"Print 'Passed' for students whose score is above 70."

```

-- Project: University Admission Management System
-- Semester: Spring 2024-2025
-- Course: Advanced Database Management Systems

```

-- Section: A

BEGIN

FOR r IN (SELECT student_id, total_marks FROM Student) LOOP

IF r.total_marks > 70 THEN

DBMS_OUTPUT.PUT_LINE('Student ' || r.student_id || ' Passed');

END IF;

END LOOP;

END;

/

6. Conditional Statements

Question 1:

"Check if the student's result status is 'Accepted'."

-- Project: University Admission Management System

-- Semester: Spring 2024-2025

-- Course: Advanced Database Management Systems

-- Section: A

DECLARE

v_result_status VARCHAR2(50);

BEGIN

SELECT result_status INTO v_result_status

FROM Application

WHERE student_id = 1;

IF v_result_status = 'Accepted' THEN

DBMS_OUTPUT.PUT_LINE('Student is Accepted.');

ELSE

DBMS_OUTPUT.PUT_LINE('Student is Not Accepted.');

END IF;

END;

/

Question 2:

"Determine if a student has scored more than 60 marks."

-- Project: University Admission Management System

-- Semester: Spring 2024-2025

-- Course: Advanced Database Management Systems

-- Section: A

DECLARE

```

v_score NUMBER := 65;
BEGIN
  IF v_score > 60 THEN
    DBMS_OUTPUT.PUT_LINE('Student has scored more than 60 marks.');
```

ELSE

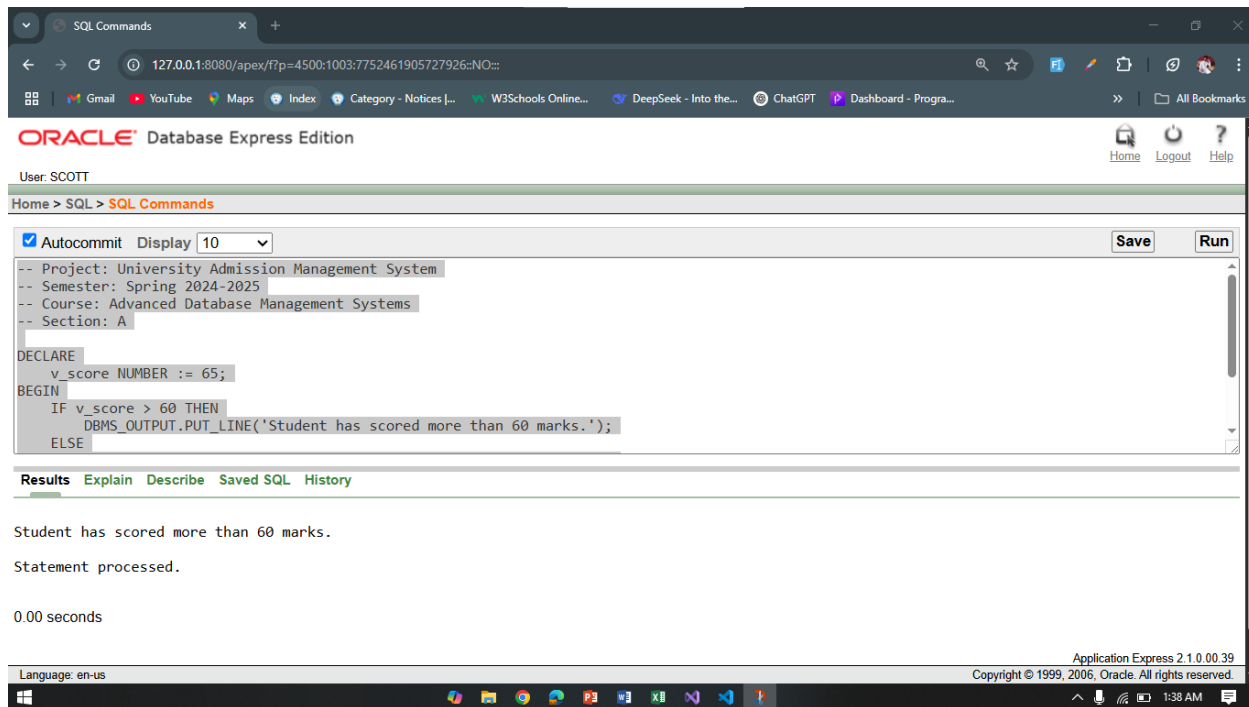
```

    DBMS_OUTPUT.PUT_LINE('Student has scored less than 60 marks.');
```

END IF;

```

END;
/
```



7. Subquery

Question 1:

"Retrieve the student ID of the student with the highest marks."

```

-- Project: University Admission Management System
-- Semester: Spring 2024-2025
-- Course: Advanced Database Management Systems
-- Section: A
```

```

DECLARE
  v_top_student_id NUMBER;
BEGIN
  SELECT student_id
  INTO v_top_student_id
  FROM Student
  WHERE student_id = (SELECT student_id FROM Test WHERE total_marks = (SELECT
    MAX(total_marks) FROM Test));

  DBMS_OUTPUT.PUT_LINE('Top Student ID: ' || v_top_student_id);
```

END;

/

Question 2:

"Find the student_id for students who have a result_status of 'Accepted'."

-- Project: University Admission Management System

-- Semester: Spring 2024-2025

-- Course: Advanced Database Management Systems

-- Section: A

DECLARE

v_student_id NUMBER;

BEGIN

SELECT student_id

INTO v_student_id

FROM Application

WHERE result_status = 'Accepted'

AND ROWNUM = 1;

DBMS_OUTPUT.PUT_LINE('Student ID of Accepted Applicant: ' || v_student_id);

END;

/

8. Joining

Question 1:

"Join Student and Application to display student name and application status."

-- Project: University Admission Management System

-- Semester: Spring 2024-2025

-- Course: Advanced Database Management Systems

-- Section: A

BEGIN

FOR r IN (SELECT s.name, a.result_status

FROM Student s

JOIN Application a ON s.student_id = a.student_id) LOOP

DBMS_OUTPUT.PUT_LINE('Student: ' || r.name || ' - Status: ' || r.result_status);

END LOOP;

END;

/

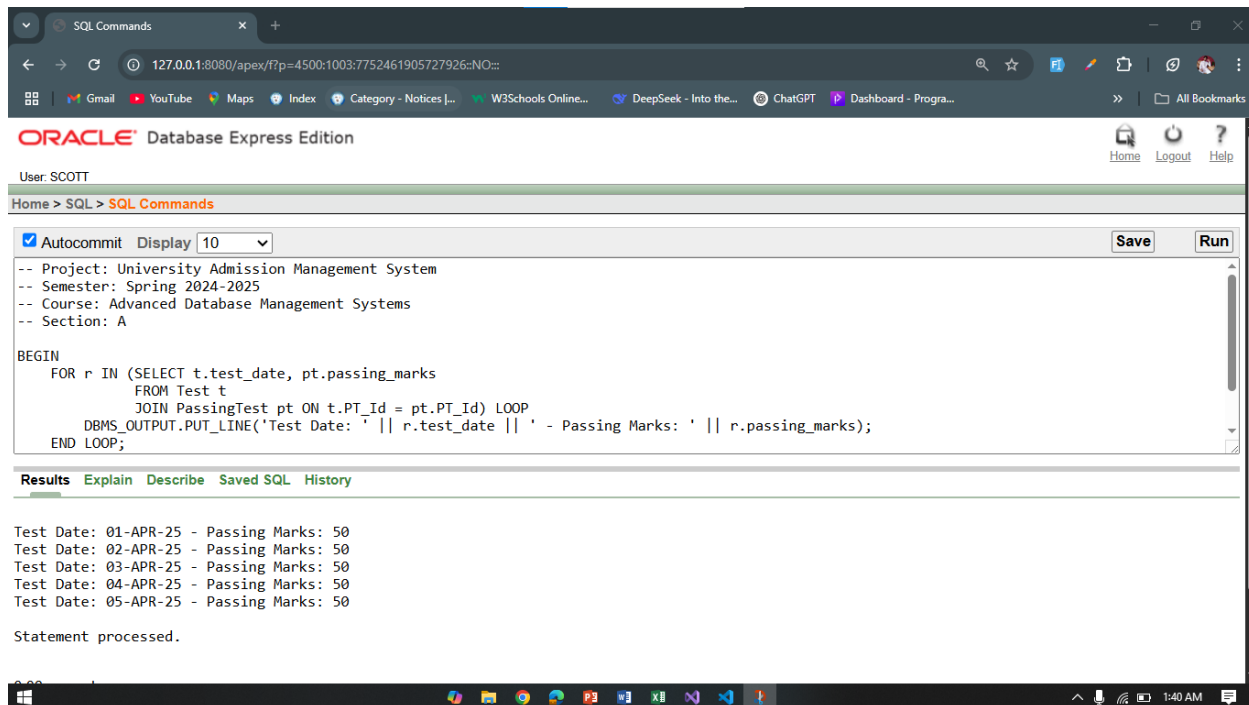
Question 2:

"Join Test and PassingTest to display test date and passing marks."

-- Project: University Admission Management System

```
-- Semester: Spring 2024-2025
-- Course: Advanced Database Management Systems
-- Section: A
```

```
BEGIN
  FOR r IN (SELECT t.test_date, pt.passing_marks
            FROM Test t
            JOIN PassingTest pt ON t.PT_Id = pt.PT_Id) LOOP
    DBMS_OUTPUT.PUT_LINE('Test Date: ' || r.test_date || ' - Passing Marks: ' || r.passing_marks);
  END LOOP;
END;
/
```



Advanced PL/SQL

1. Stored Functions

Question 1:

"Create a function to retrieve the result status of a student based on their student_id."

```
-- Project: University Admission Management System
-- Semester: Spring 2024-2025
-- Course: Advanced Database Management Systems
-- Section: A
```

```
CREATE OR REPLACE FUNCTION get_result_status (p_student_id IN NUMBER)
RETURN VARCHAR2 IS
  v_result_status VARCHAR2(50);
BEGIN
  SELECT result_status
```

```
    INTO v_result_status
  FROM Application
 WHERE student_id = p_student_id;
```

```
    RETURN v_result_status;
END get_result_status;
/
```

Question 2:

"Create a function to calculate the total marks of a student in all tests."

-- Project: University Admission Management System
-- Semester: Spring 2024-2025
-- Course: Advanced Database Management Systems
-- Section: A

```
CREATE OR REPLACE FUNCTION get_total_marks (p_student_id IN NUMBER)
RETURN NUMBER IS
  v_total_marks NUMBER := 0;
BEGIN
  SELECT SUM(total_marks)
  INTO v_total_marks
  FROM Test
  WHERE student_id = p_student_id;

  RETURN v_total_marks;
END get_total_marks;
/
```

2. Stored Procedures

Question 1:

"Create a stored procedure to update the result status of a student based on their student_id."

-- Project: University Admission Management System
-- Semester: Spring 2024-2025
-- Course: Advanced Database Management Systems
-- Section: A

```
CREATE OR REPLACE PROCEDURE update_result_status (p_student_id IN NUMBER, p_status IN
VARCHAR2)
IS
BEGIN
  UPDATE Application
  SET result_status = p_status
  WHERE student_id = p_student_id;
```

```
COMMIT;
END update_result_status;
/
```

Question 2:

"Create a stored procedure to add a new student into the Student table."

```
-- Project: University Admission Management System
-- Semester: Spring 2024-2025
-- Course: Advanced Database Management Systems
-- Section: A
```

```
CREATE OR REPLACE PROCEDURE add_student (p_name IN VARCHAR2, p_phone IN
VARCHAR2, p_email IN VARCHAR2, p_city IN VARCHAR2)
```

```
IS
```

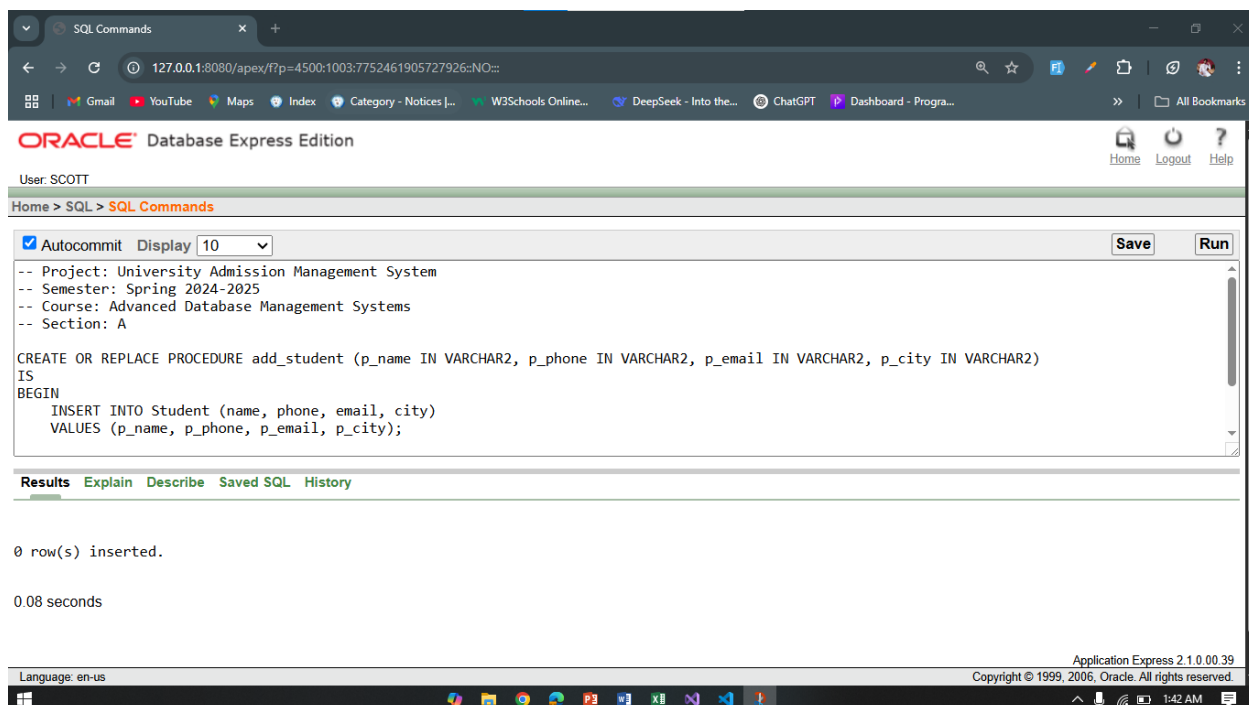
```
BEGIN
```

```
    INSERT INTO Student (name, phone, email, city)
    VALUES (p_name, p_phone, p_email, p_city);
```

```
COMMIT;
```

```
END add_student;
```

```
/
```



3. Table-Based Record

Question 1:

"Create a table-based record type to store student data and loop through it."

```
-- Project: University Admission Management System
-- Semester: Spring 2024-2025
-- Course: Advanced Database Management Systems
```


-- Section: A

DECLARE

TYPE student_record_type IS TABLE OF Student%ROWTYPE;

v_students student_record_type;

BEGIN

SELECT * BULK COLLECT INTO v_students FROM Student;

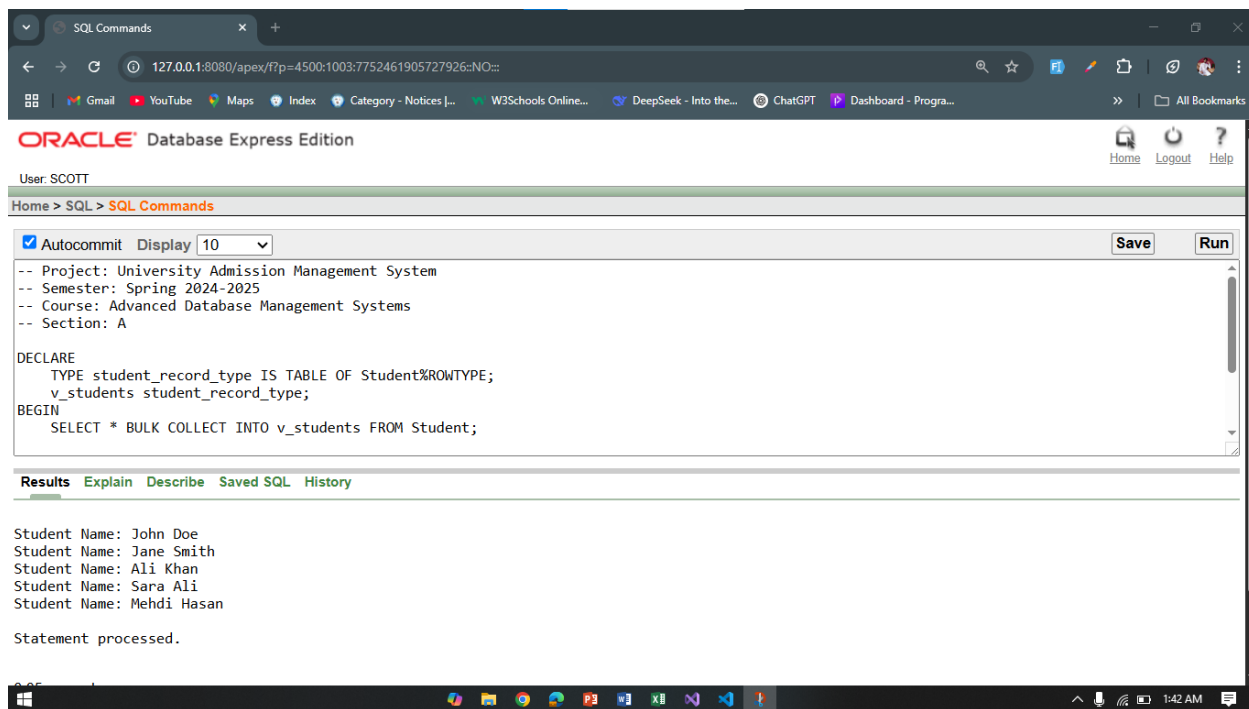
FOR i IN 1..v_students.COUNT LOOP

DBMS_OUTPUT.PUT_LINE('Student Name: ' || v_students(i).name);

END LOOP;

END;

/



Question 2:

"Create a table-based record to collect test data and print test dates."

-- Project: University Admission Management System

-- Semester: Spring 2024-2025

-- Course: Advanced Database Management Systems

-- Section: A

DECLARE

TYPE test_record_type IS TABLE OF Test%ROWTYPE;

v_tests test_record_type;

BEGIN

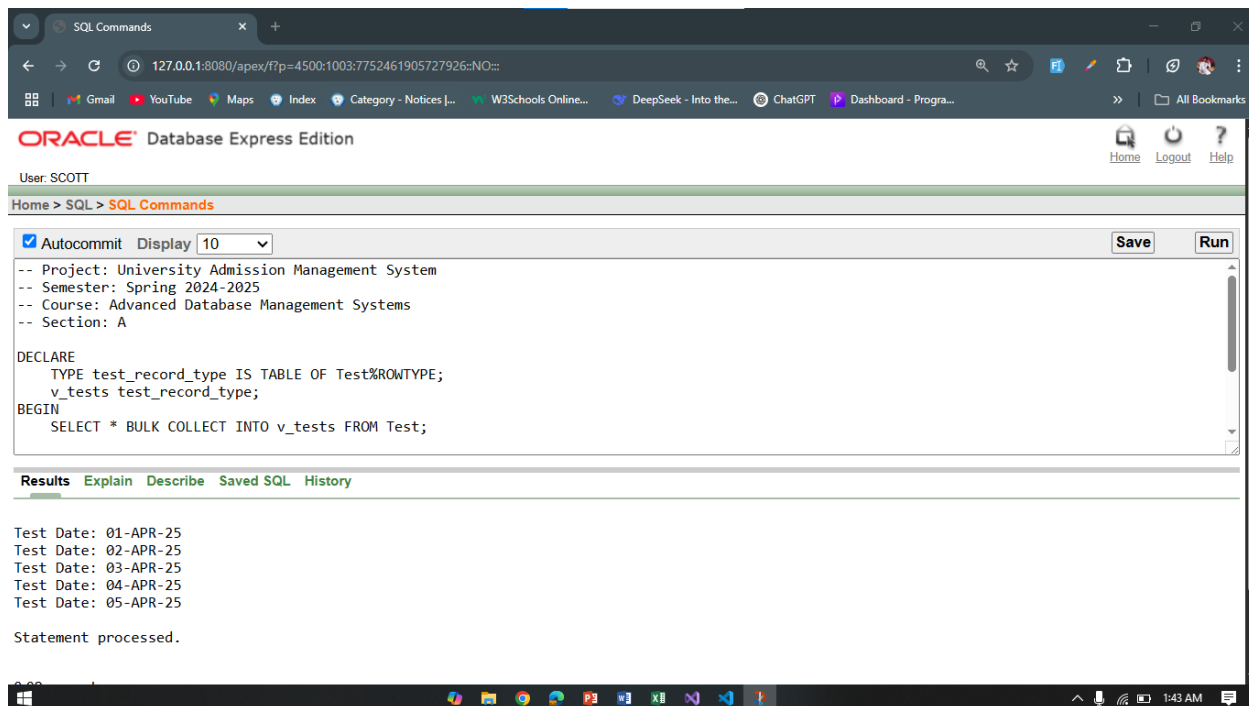
SELECT * BULK COLLECT INTO v_tests FROM Test;

FOR i IN 1..v_tests.COUNT LOOP

```

        DBMS_OUTPUT.PUT_LINE('Test Date: ' || v_tests(i).test_date);
    END LOOP;
END;
/

```



4. Explicit Cursor

Question 1:

"Create an explicit cursor to fetch students with their test scores."

```

-- Project: University Admission Management System
-- Semester: Spring 2024-2025
-- Course: Advanced Database Management Systems
-- Section: A

```

```

DECLARE

```

```

    CURSOR c_students IS
        SELECT s.student_id, t.total_marks
        FROM Student s
        JOIN Test t ON s.student_id = t.student_id;

```

```

    v_student c_students%ROWTYPE;

```

```

BEGIN

```

```

    OPEN c_students;

```

```

    LOOP

```

```

        FETCH c_students INTO v_student;

```

```

        EXIT WHEN c_students%NOTFOUND;

```

```

        DBMS_OUTPUT.PUT_LINE('Student ID: ' || v_student.student_id || ', Total Marks: ' ||
v_student.total_marks);

```

```

    END LOOP;

```

```
CLOSE c_students;
END;
/
```

Question 2:

"Create an explicit cursor to fetch students who passed the test."

```
-- Project: University Admission Management System
-- Semester: Spring 2024-2025
-- Course: Advanced Database Management Systems
-- Section: A
```

```
DECLARE
```

```
CURSOR c_passed_students IS
  SELECT s.student_id, t.total_marks
  FROM Student s
  JOIN Test t ON s.student_id = t.student_id
  WHERE t.total_marks >= 50;
```

```
v_student c_passed_students%ROWTYPE;
BEGIN
  OPEN c_passed_students;
  LOOP
    FETCH c_passed_students INTO v_student;
    EXIT WHEN c_passed_students%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE('Student ID: ' || v_student.student_id || ', Total Marks: ' ||
v_student.total_marks);
  END LOOP;
  CLOSE c_passed_students;
END;
/
```

5. Cursor-Based Record

Question 1:

"Use a cursor-based record to print the names of students who passed the test."

```
-- Project: University Admission Management System
-- Semester: Spring 2024-2025
-- Course: Advanced Database Management Systems
-- Section: A
```

```
DECLARE
```

```
TYPE student_rec_type IS RECORD (
  student_id NUMBER,
  name VARCHAR2(100)
```

```

);
TYPE student_tab_type IS TABLE OF student_rec_type;
v_students student_tab_type;
BEGIN
    SELECT student_id, name
    BULK COLLECT INTO v_students
    FROM Student
    WHERE student_id IN (SELECT student_id FROM Test WHERE total_marks >= 50);

    FOR i IN 1..v_students.COUNT LOOP
        DBMS_OUTPUT.PUT_LINE('Student Name: ' || v_students(i).name);
    END LOOP;
END;
/

```

Question 2:

"Fetch student names and display using cursor-based record."

```

-- Project: University Admission Management System
-- Semester: Spring 2024-2025
-- Course: Advanced Database Management Systems
-- Section: A

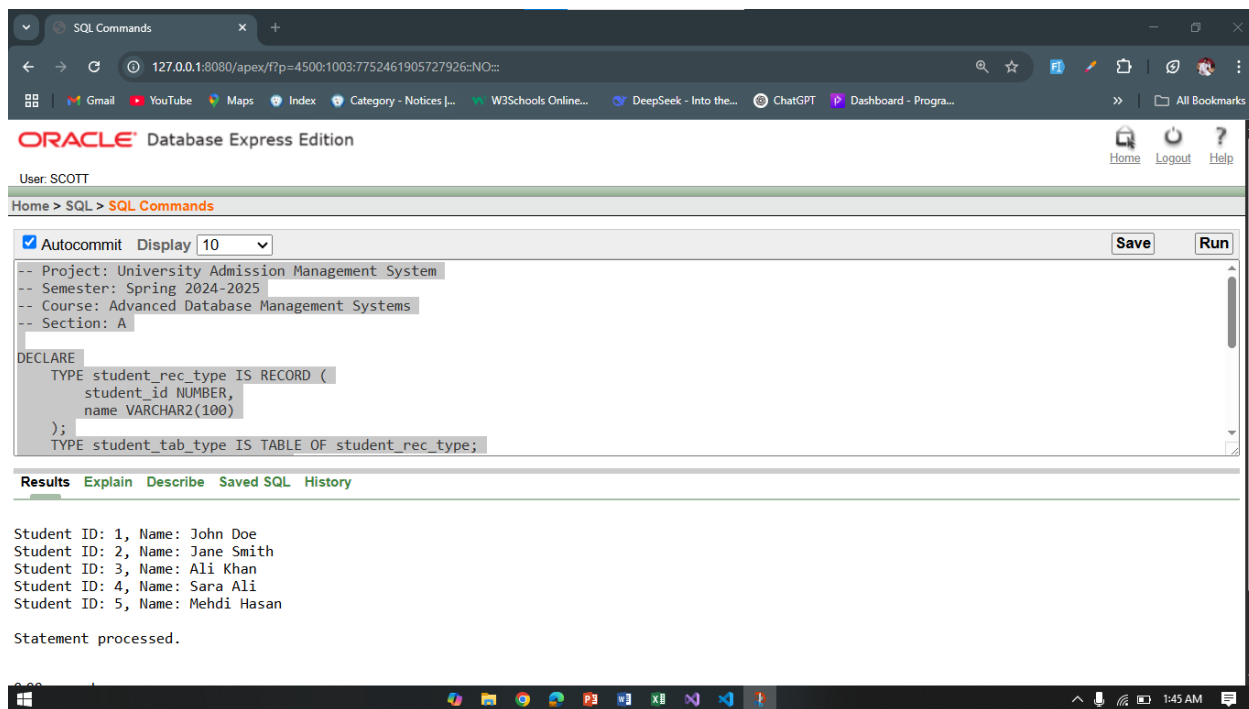
```

```

DECLARE
    TYPE student_rec_type IS RECORD (
        student_id NUMBER,
        name VARCHAR2(100)
    );
    TYPE student_tab_type IS TABLE OF student_rec_type;
    v_students student_tab_type;
BEGIN
    SELECT student_id, name
    BULK COLLECT INTO v_students
    FROM Student;

    FOR i IN 1..v_students.COUNT LOOP
        DBMS_OUTPUT.PUT_LINE('Student ID: ' || v_students(i).student_id || ', Name: ' ||
v_students(i).name);
    END LOOP;
END;
/

```



6. Row-Level Trigger

Question 1:

"Create a row-level trigger that updates the email to uppercase when a student record is inserted."

-- Project: University Admission Management System

-- Semester: Spring 2024-2025

-- Course: Advanced Database Management Systems

-- Section: A

CREATE OR REPLACE TRIGGER update_email_trigger

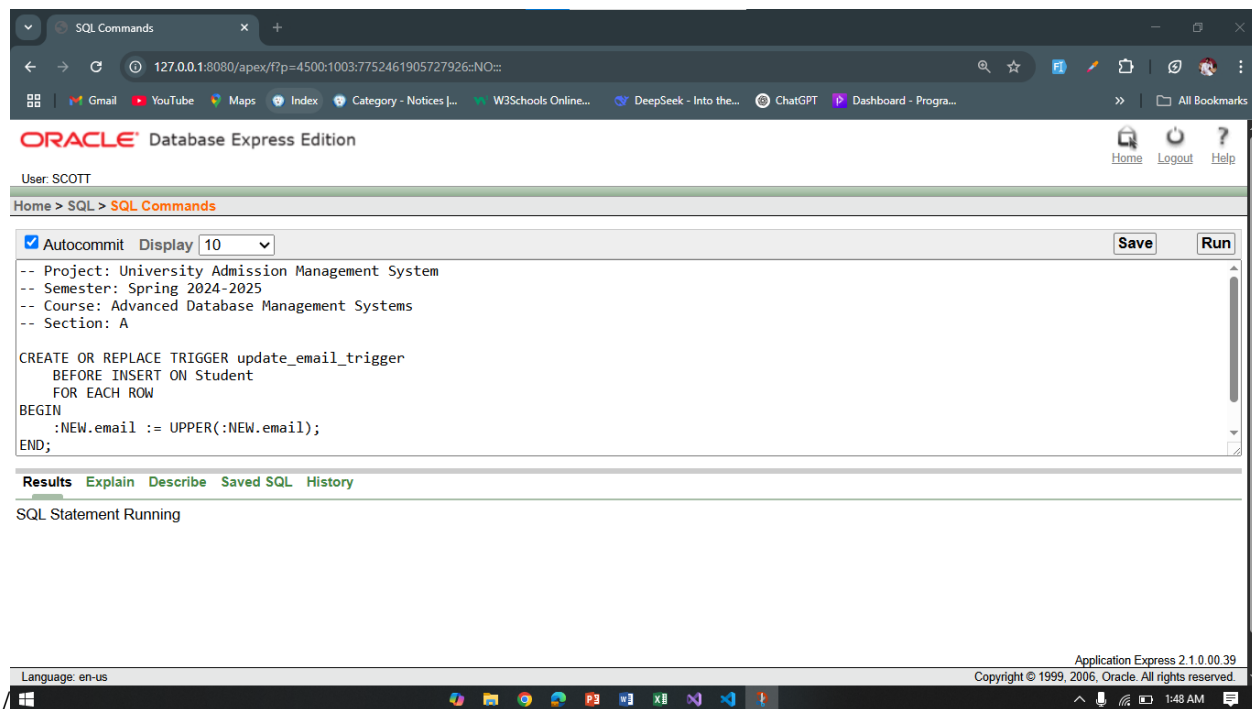
BEFORE INSERT ON Student

FOR EACH ROW

BEGIN

:NEW.email := UPPER(:NEW.email);

END;



Question 2:

"Create a row-level trigger to log changes made to student names."

```
CREATE OR REPLACE TRIGGER log_student_name_changes
  BEFORE UPDATE ON Student
  FOR EACH ROW
BEGIN
  INSERT INTO LogTable (log_message)
  VALUES ('Student Name Changed from ' || :OLD.name || ' to ' || :NEW.name);
END;
```

7. Statement-Level Trigger

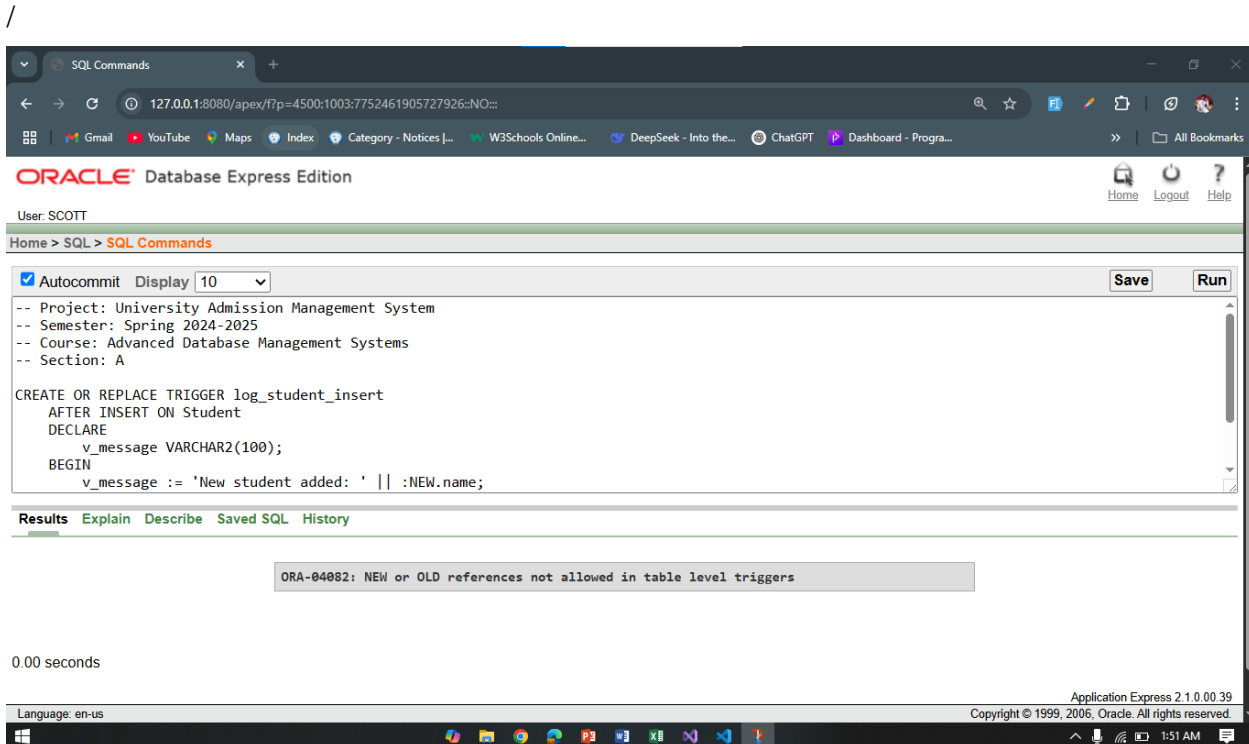
Question 1:

"Create a statement-level trigger to log a message whenever a new student record is inserted."

```
-- Project: University Admission Management System
-- Semester: Spring 2024-2025
-- Course: Advanced Database Management Systems
-- Section: A
```

```
CREATE OR REPLACE TRIGGER log_student_insert
  AFTER INSERT ON Student
  DECLARE
    v_message VARCHAR2(100);
  BEGIN
```

```
v_message := 'New student added: ' || :NEW.name;
INSERT INTO LogTable (log_message) VALUES (v_message);
END;
```

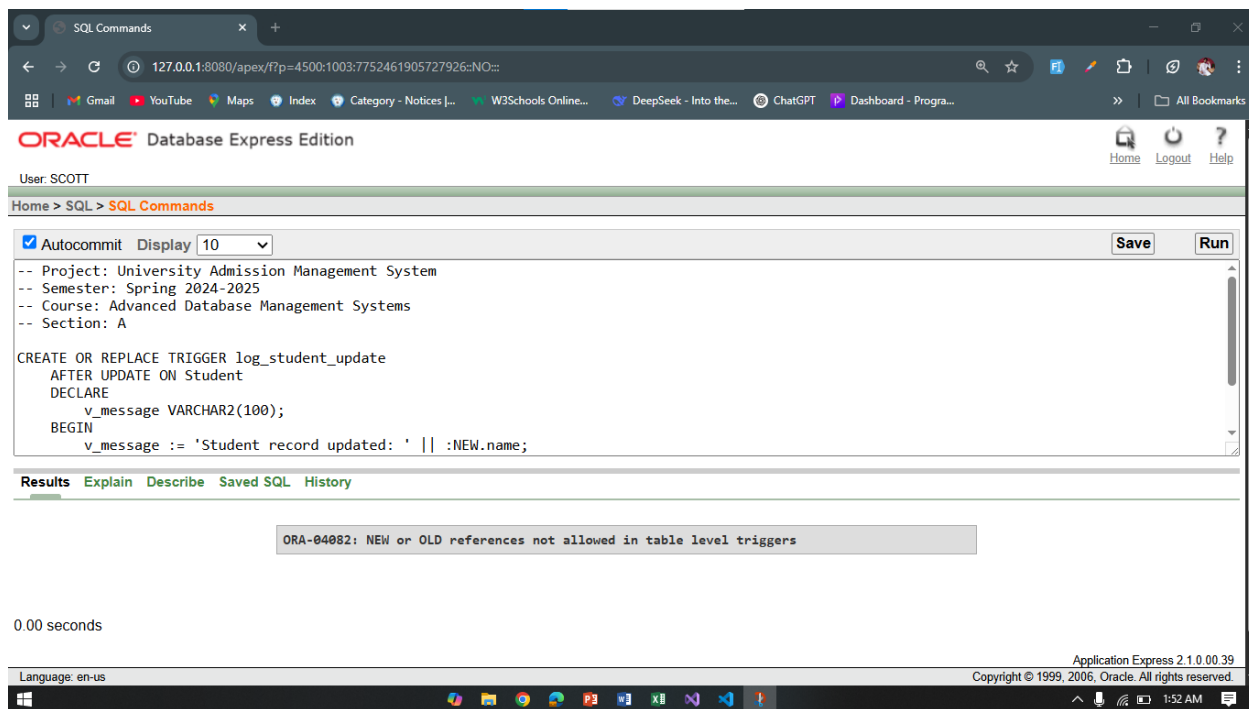


Question 2:

"Create a statement-level trigger to log a message after updating student records."

```
-- Project: University Admission Management System
-- Semester: Spring 2024-2025
-- Course: Advanced Database Management Systems
-- Section: A
```

```
CREATE OR REPLACE TRIGGER log_student_update
AFTER UPDATE ON Student
DECLARE
    v_message VARCHAR2(100);
BEGIN
    v_message := 'Student record updated: ' || :NEW.name;
    INSERT INTO LogTable (log_message) VALUES (v_message);
END;
```



8. Package

Question 1:

"Create a package to manage student records, including procedures to add and update students."

```
-- Project: University Admission Management System
```

-- Semester: Spring 2024-2025

-- Course: Advanced Database Management Systems

-- Section: A

CREATE OR REPLACE PACKAGE student_pkg AS

```
PROCEDURE add_student(p_name VARCHAR2, p_phone VARCHAR2, p_email VARCHAR2);
```

```
PROCEDURE update_student_status(p_student_id NUMBER, p_status VARCHAR2);
```

END student_pkg;

/

CREATE OR REPLACE PACKAGE BODY student_pkg AS

```
PROCEDURE add_student(p_name VARCHAR2, p_phone VARCHAR2, p_email VARCHAR2) IS
```

BEGIN

```
INSERT INTO Student (name, phone, email) VALUES (p_name, p_phone, p_email);
```

END add_student;

```
PROCEDURE update_student_status(p_student_id NUMBER, p_status VARCHAR2) IS
```

BEGIN

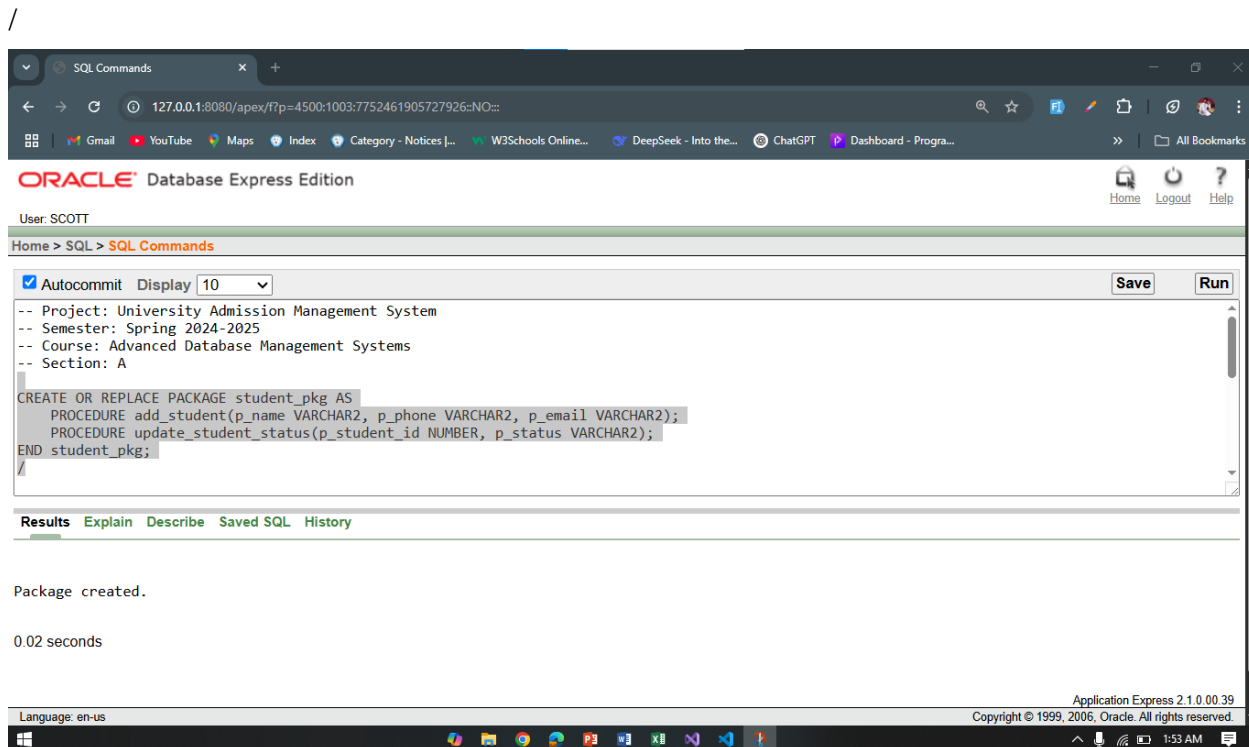
UPDATE Application

```
SET result_status = p_status
```

WHERE student_id = p_student_id;

END update_student_status;

END student_pkg;



Question 2:

"Create a package to handle student test results and calculate their performance."

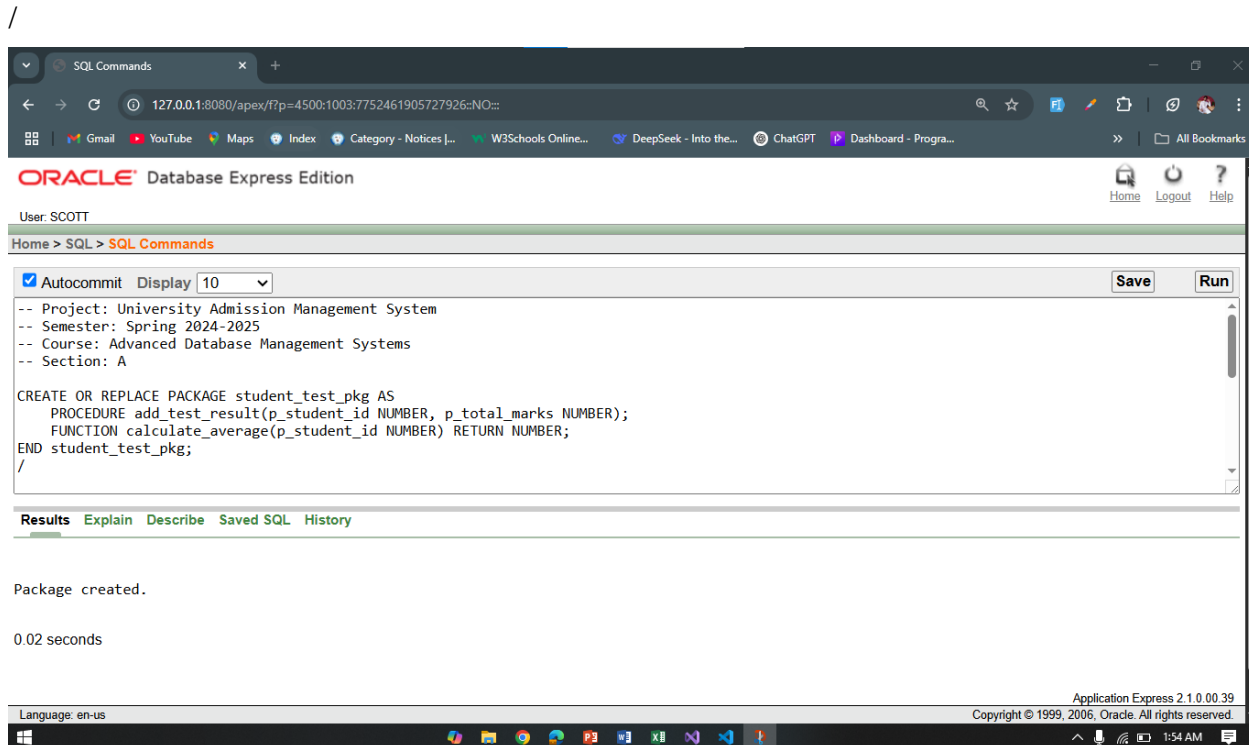
- Project: University Admission Management System
- Semester: Spring 2024-2025
- Course: Advanced Database Management Systems
- Section: A

```
CREATE OR REPLACE PACKAGE student_test_pkg AS
    PROCEDURE add_test_result(p_student_id NUMBER, p_total_marks NUMBER);
    FUNCTION calculate_average(p_student_id NUMBER) RETURN NUMBER;
END student_test_pkg;
```

/

```
CREATE OR REPLACE PACKAGE BODY student_test_pkg AS
    PROCEDURE add_test_result(p_student_id NUMBER, p_total_marks NUMBER) IS
    BEGIN
        INSERT INTO Test (student_id, total_marks) VALUES (p_student_id, p_total_marks);
    END add_test_result;

    FUNCTION calculate_average(p_student_id NUMBER) RETURN NUMBER IS
        v_avg NUMBER;
    BEGIN
        SELECT AVG(total_marks) INTO v_avg FROM Test WHERE student_id = p_student_id;
        RETURN v_avg;
    END calculate_average;
END student_test_pkg;
```



Conclusion:

The project successfully implements a robust DBMS for handling student data, applications, test results, and staff management, leveraging PL/SQL for advanced business logic. By integrating optimization techniques, improving user interface design, enhancing security, and exploring future integrations like machine learning or cloud computing, the system can evolve into a fully scalable, secure, and user-friendly application capable of meeting the growing needs of educational institutions.